

In []: Submitted By

```
**Name:** Rajnish Kumar Sadhan
**Course:** Bachelor of Technology (Computer Science & Engineering)
**Specialization:** Artificial Intelligence | Machine Learning | Data Science
**Email:** kumarsadhanrajnish@gmail.com
**Phone:** +91 8102655393
```

Submitted To

```
**Accredian**
**Role:** Data Science Intern
```

In []: Project Overview

This project focuses on analyzing instructor effectiveness using educational data. The project includes comprehensive Exploratory Data Analysis (EDA), statistical analysis, and machine learning models for classification and regression. ---

In []: #  Table of Contents

1. Import Required Libraries
2. Load Dataset
3. Dataset Understanding
4. Data Cleaning
5. Exploratory Data Analysis (EDA)
6. Statistical Analysis
7. Correlation Analysis
8. Feature Engineering
9. Principal Component Analysis (PCA)
10. K-Means Clustering
11. Interactive Visualizations
12. Business Insights
13. Recommendations
14. Conclusion
15. Future Scope

```
In [56]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

import warnings
warnings.filterwarnings("ignore")

plt.style.use("ggplot")
plt.rcParams["figure.figsize"] = (10,6)
pd.set_option("display.max_columns", None)
```

```
print("Libraries Imported Successfully")
```

Libraries Imported Successfully

```
In [58]: df = pd.read_csv("instructor_effectiveness_dataset_2000_rows - instructor_ef
```

```
In [59]: df.head()
```

```
Out[59]:
```

	batch_id	instructor_id	course_id	completion_rate	avg_score_improvement	avg_qu
0	B_1861	I_044	C_01	0.300000	14.225955	73
1	B_0354	I_119	C_06	0.657220	22.871110	77
2	B_1334	I_050	C_03	0.300000	16.087517	79
3	B_0906	I_024	C_21	0.639507	24.260687	99
4	B_1290	I_001	C_08	0.527302	31.081556	99

```
In [60]: df.tail()
```

```
Out[60]:
```

	batch_id	instructor_id	course_id	completion_rate	avg_score_improvement	avg
1995	B_1131	I_047	C_17	0.589309	19.227979	
1996	B_1295	I_037	C_25	0.980000	36.737265	
1997	B_0861	I_033	C_06	0.531447	24.907878	
1998	B_1460	I_069	C_10	0.641025	22.395362	
1999	B_1127	I_045	C_22	0.300000	23.101794	

```
In [61]: df.sample(5, random_state=42)
```

```
Out[61]:
```

	batch_id	instructor_id	course_id	completion_rate	avg_score_improvement	avg
1860	B_1523	I_040	C_18	0.632279	29.964500	
353	B_1611	I_114	C_12	0.659228	40.000000	
1333	B_0569	I_063	C_10	0.306751	27.333369	
905	B_1802	I_040	C_23	0.505999	25.732181	
1289	B_0091	I_054	C_25	0.687284	28.941081	

```
In [62]: print("Rows :",df.shape[0])  
print("Columns :",df.shape[1])
```

Rows : 2000
Columns : 12

```
In [64]: for col in df.columns:
         print(col)
```

```
batch_id
instructor_id
course_id
completion_rate
avg_score_improvement
avg_quiz_score
dropout_rate
avg_watch_time
assignment_submission_rate
forum_activity_rate
avg_feedback_score
feedback_response_rate
```

```
In [65]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   batch_id                             2000 non-null   str
1   instructor_id                         2000 non-null   str
2   course_id                             2000 non-null   str
3   completion_rate                       2000 non-null   float64
4   avg_score_improvement                 2000 non-null   float64
5   avg_quiz_score                        2000 non-null   float64
6   dropout_rate                          2000 non-null   float64
7   avg_watch_time                        2000 non-null   float64
8   assignment_submission_rate             2000 non-null   float64
9   forum_activity_rate                   2000 non-null   float64
10  avg_feedback_score                     2000 non-null   float64
11  feedback_response_rate                 2000 non-null   float64
dtypes: float64(9), str(3)
memory usage: 187.6 KB
```

```
In [66]: df.dtypes
```

```
Out[66]: batch_id          str
instructor_id         str
course_id             str
completion_rate       float64
avg_score_improvement float64
avg_quiz_score        float64
dropout_rate          float64
avg_watch_time        float64
assignment_submission_rate float64
forum_activity_rate   float64
avg_feedback_score    float64
feedback_response_rate float64
dtype: object
```

```
In [67]: df.isnull().sum()
```

```
Out[67]: batch_id          0
instructor_id          0
course_id              0
completion_rate        0
avg_score_improvement  0
avg_quiz_score         0
dropout_rate           0
avg_watch_time         0
assignment_submission_rate 0
forum_activity_rate    0
avg_feedback_score     0
feedback_response_rate 0
dtype: int64
```

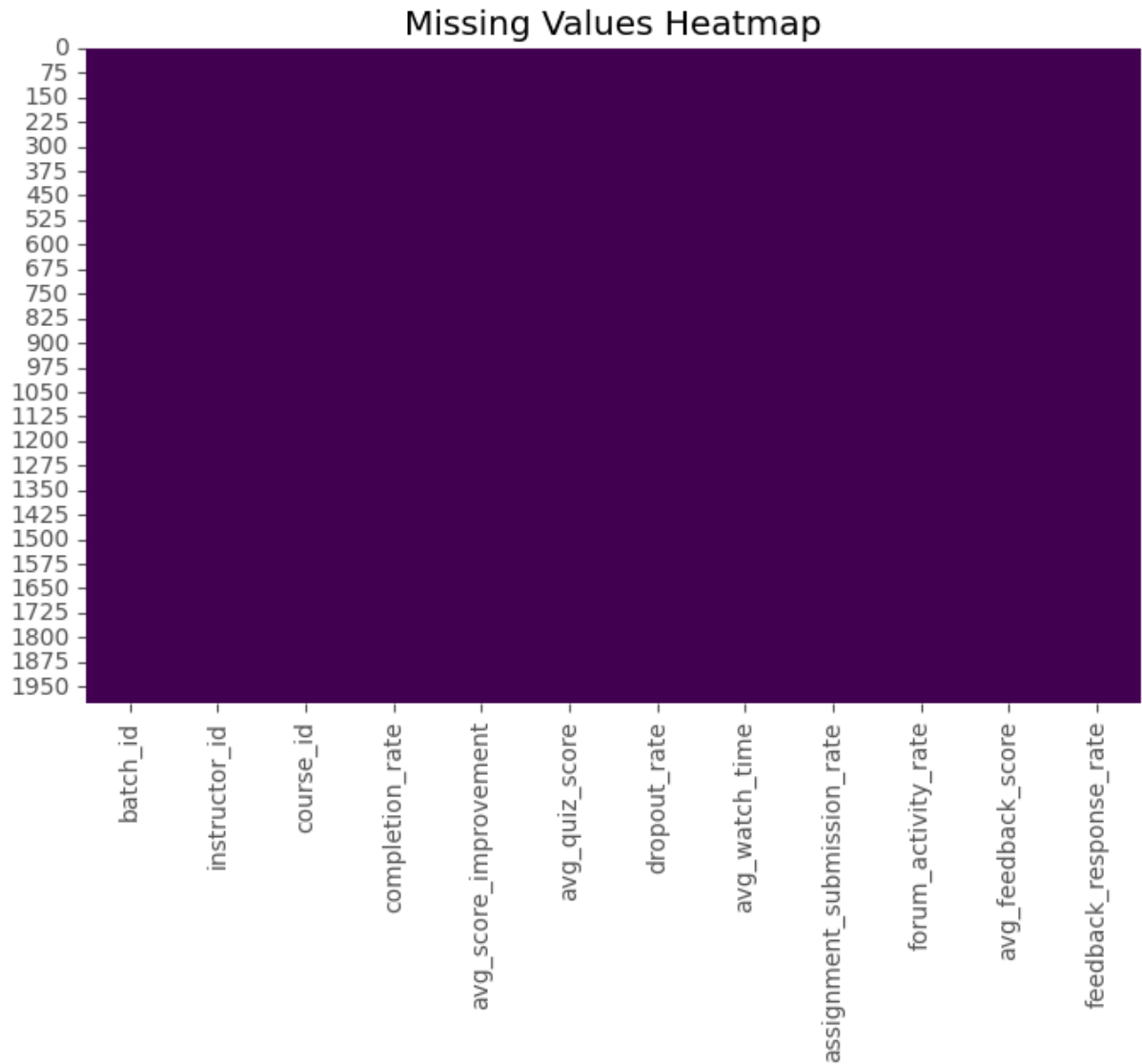
```
In [68]: missing = pd.DataFrame({
    "Missing Values":df.isnull().sum(),
    "Percentage":round(df.isnull().mean()*100,2)
})
missing
```

```
Out[68]:
```

	Missing Values	Percentage
batch_id	0	0.0
instructor_id	0	0.0
course_id	0	0.0
completion_rate	0	0.0
avg_score_improvement	0	0.0
avg_quiz_score	0	0.0
dropout_rate	0	0.0
avg_watch_time	0	0.0
assignment_submission_rate	0	0.0
forum_activity_rate	0	0.0
avg_feedback_score	0	0.0
feedback_response_rate	0	0.0

```
In [69]: plt.figure(figsize=(8,5))
sns.heatmap(df.isnull(),cbar=False,cmap="viridis")
plt.title("Missing Values Heatmap")
```

```
plt.show()
```



```
In [70]: df.duplicated().sum()
```

```
Out[70]: np.int64(0)
```

```
In [71]: df = df.drop_duplicates()
```

```
In [72]: df.memory_usage()
```

```
Out[72]: Index          132
batch_id          16000
instructor_id     16000
course_id         16000
completion_rate   16000
avg_score_improvement 16000
avg_quiz_score    16000
dropout_rate      16000
avg_watch_time    16000
assignment_submission_rate 16000
forum_activity_rate 16000
avg_feedback_score 16000
feedback_response_rate 16000
dtype: int64
```

```
In [73]: df.memory_usage().sum()
```

```
Out[73]: np.int64(192132)
```

```
In [74]: df.nunique()
```

```
Out[74]: batch_id          2000
instructor_id          120
course_id              25
completion_rate        1893
avg_score_improvement  1966
avg_quiz_score         1964
dropout_rate           1907
avg_watch_time         1852
assignment_submission_rate 1876
forum_activity_rate    1987
avg_feedback_score     1934
feedback_response_rate  1907
dtype: int64
```

```
In [75]: cat_cols = df.select_dtypes(include="object").columns
num_cols = df.select_dtypes(exclude="object").columns
```

```
In [76]: print(cat_cols)
```

```
Index(['batch_id', 'instructor_id', 'course_id'], dtype='str')
```

```
In [77]: print(num_cols)
```

```
Index(['completion_rate', 'avg_score_improvement', 'avg_quiz_score',
      'dropout_rate', 'avg_watch_time', 'assignment_submission_rate',
      'forum_activity_rate', 'avg_feedback_score', 'feedback_response_rate'],
      dtype='str')
```

```
In [78]: summary = pd.DataFrame({
    "Data Type":df.dtypes,
    "Missing Values":df.isnull().sum(),
```

```

    "Unique Values":df.nunique()
})

summary

```

Out[78]:

	Data Type	Missing Values	Unique Values
batch_id	str	0	2000
instructor_id	str	0	120
course_id	str	0	25
completion_rate	float64	0	1893
avg_score_improvement	float64	0	1966
avg_quiz_score	float64	0	1964
dropout_rate	float64	0	1907
avg_watch_time	float64	0	1852
assignment_submission_rate	float64	0	1876
forum_activity_rate	float64	0	1987
avg_feedback_score	float64	0	1934
feedback_response_rate	float64	0	1907

In [79]: `summary.to_csv("Dataset_Summary.csv")`

In [80]: `print("Numerical Features:")`
`print(num_cols)`

`print("\nCategorical Features:")`
`print(cat_cols)`

```

Numerical Features:
Index(['completion_rate', 'avg_score_improvement', 'avg_quiz_score',
      'dropout_rate', 'avg_watch_time', 'assignment_submission_rate',
      'forum_activity_rate', 'avg_feedback_score', 'feedback_response_rate'],
      dtype='str')

```

```

Categorical Features:
Index(['batch_id', 'instructor_id', 'course_id'], dtype='str')

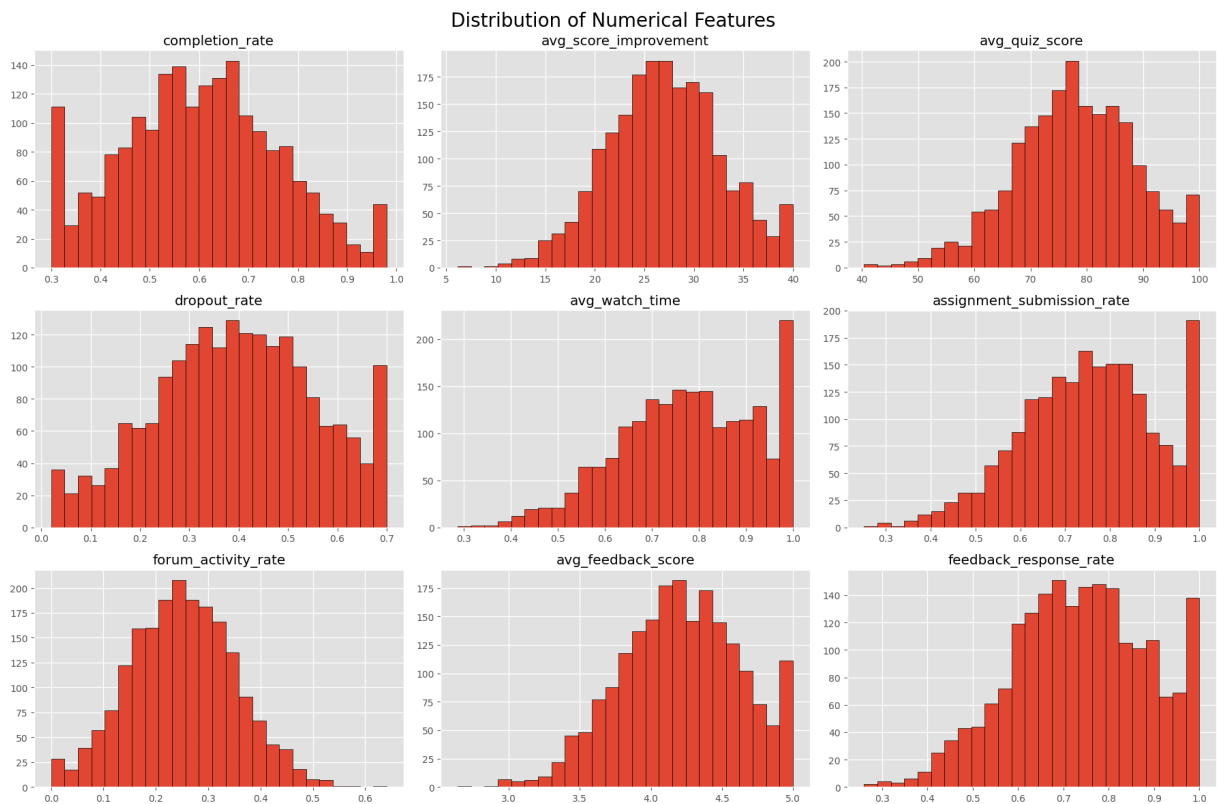
```

In [81]: `df[num_cols].hist(`
 `figsize=(18,12),`
 `bins=25,`
 `edgecolor='black'`
`)`

```
plt.suptitle("Distribution of Numerical Features", fontsize=20)

plt.tight_layout()

plt.show()
```



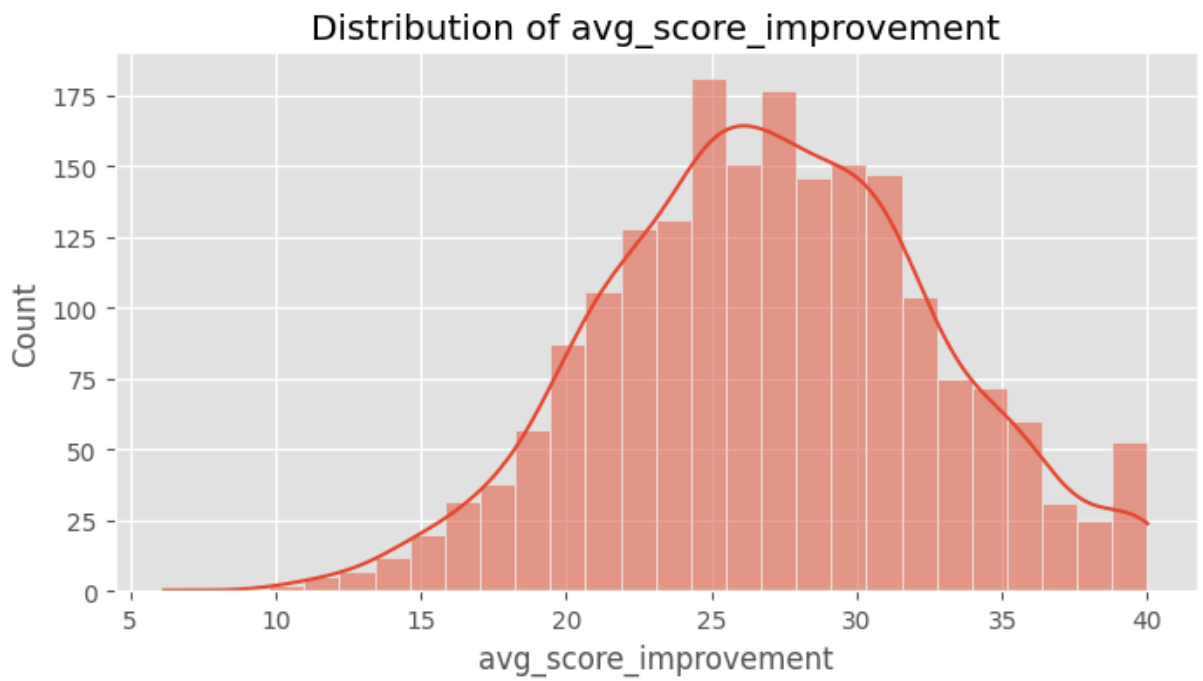
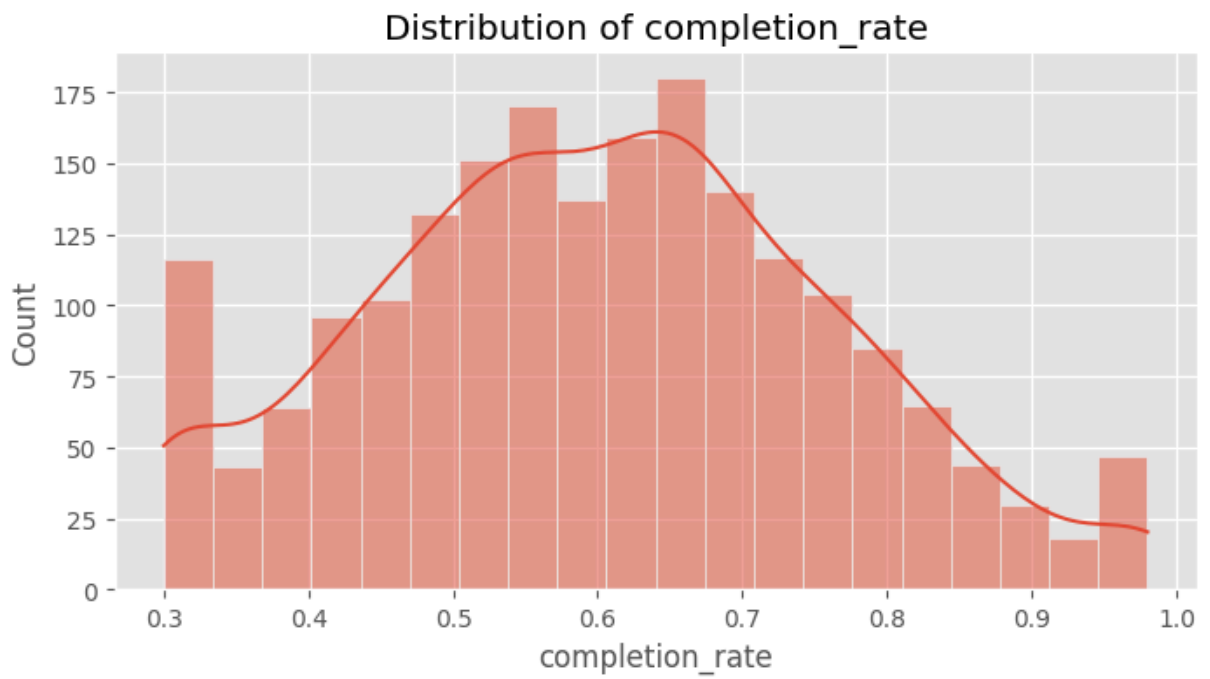
```
In [82]: for col in num_cols:

    plt.figure(figsize=(8,4))

    sns.histplot(df[col],kde=True)

    plt.title(f"Distribution of {col}")

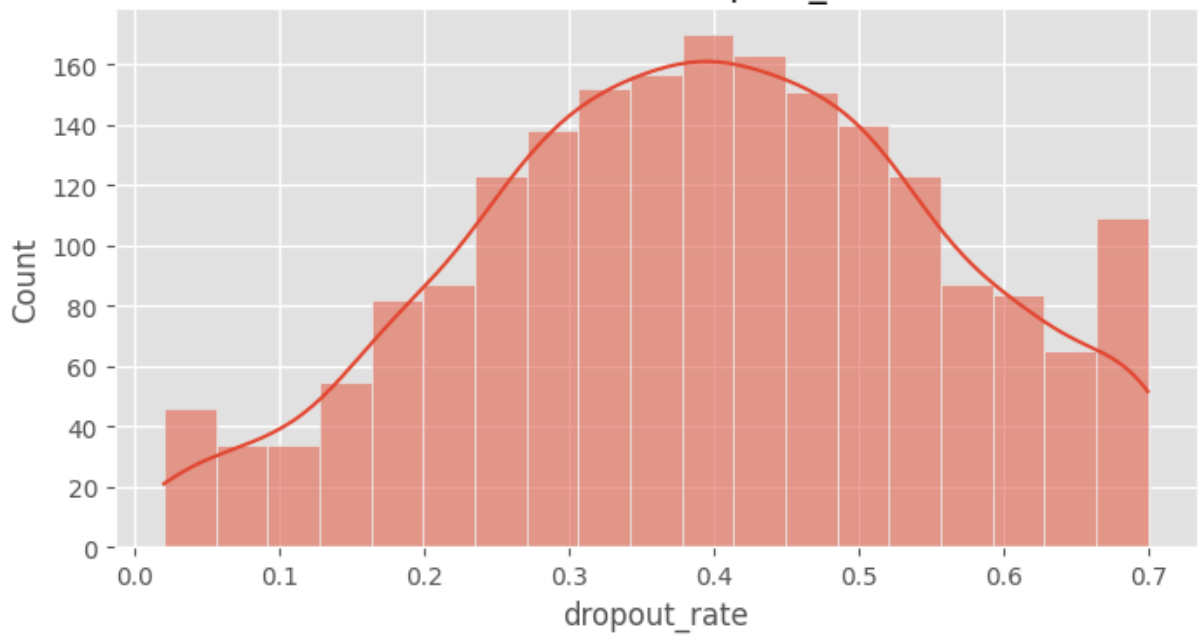
    plt.show()
```

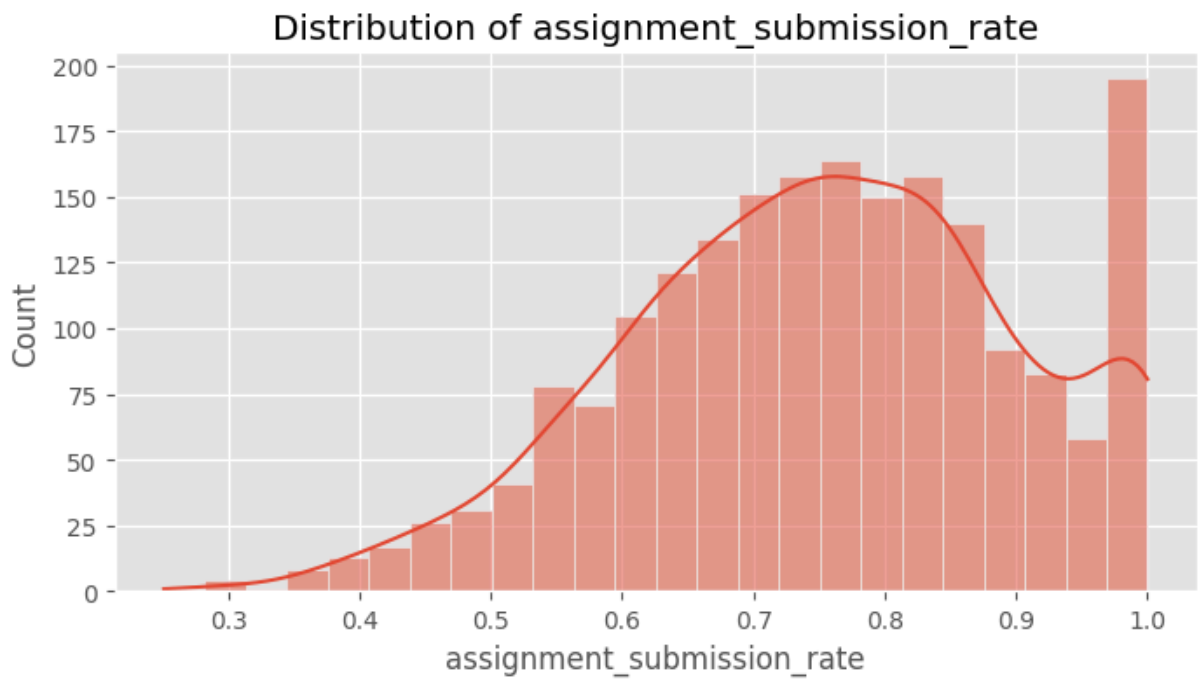
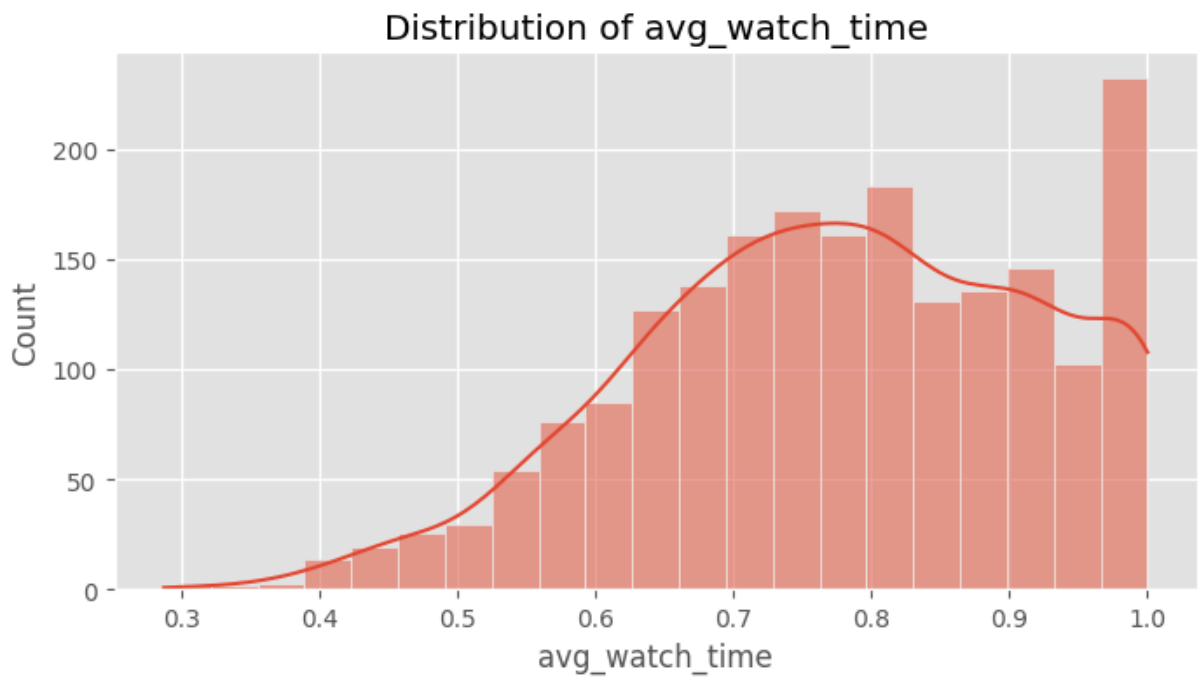



Distribution of avg_quiz_score

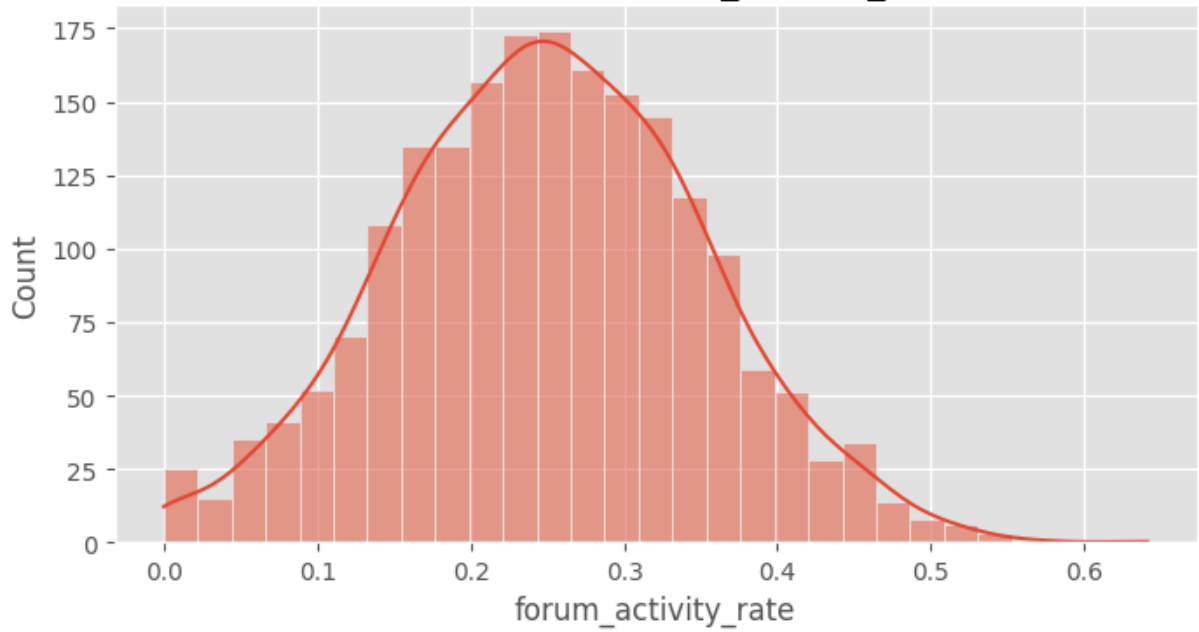


Distribution of dropout_rate

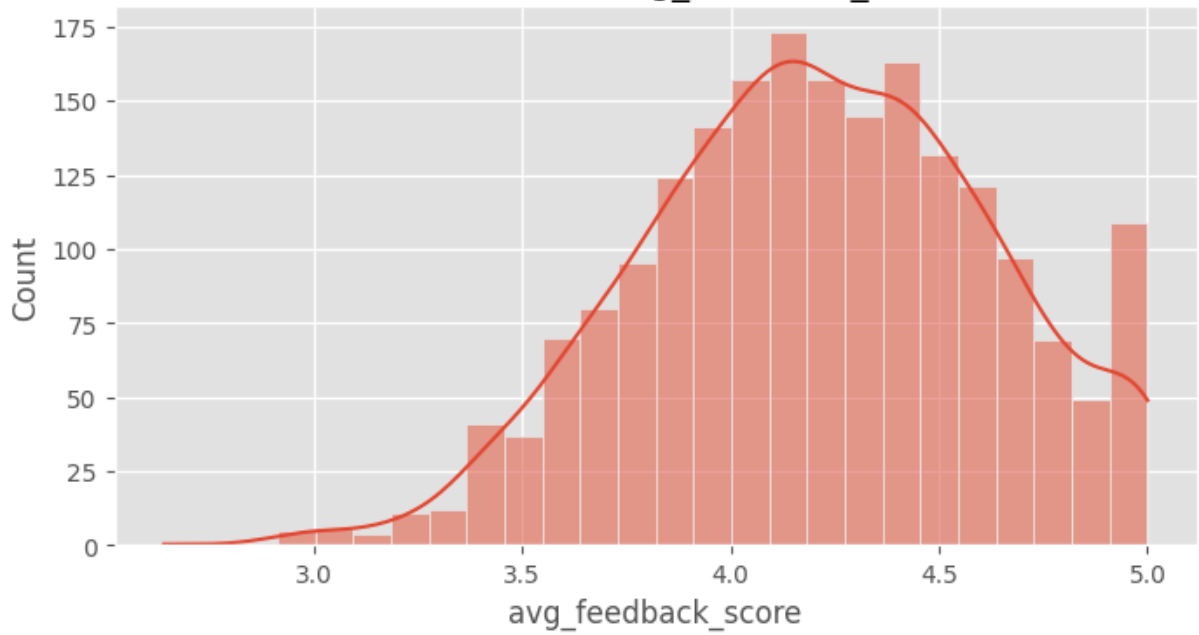


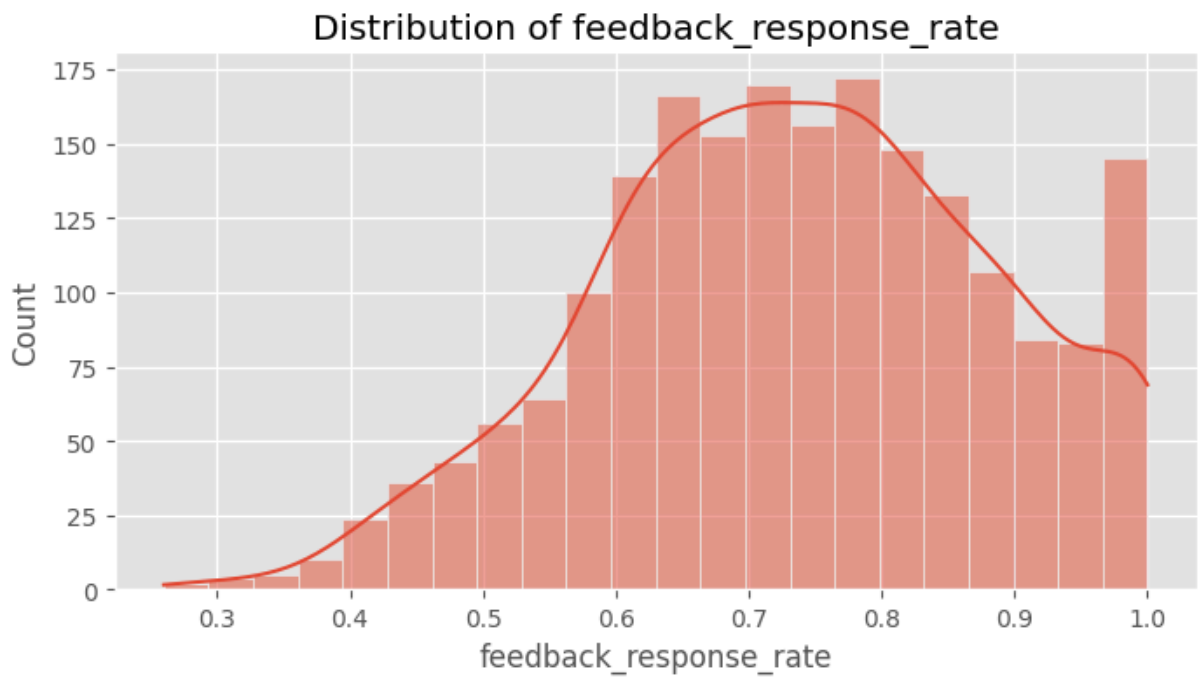


Distribution of forum_activity_rate

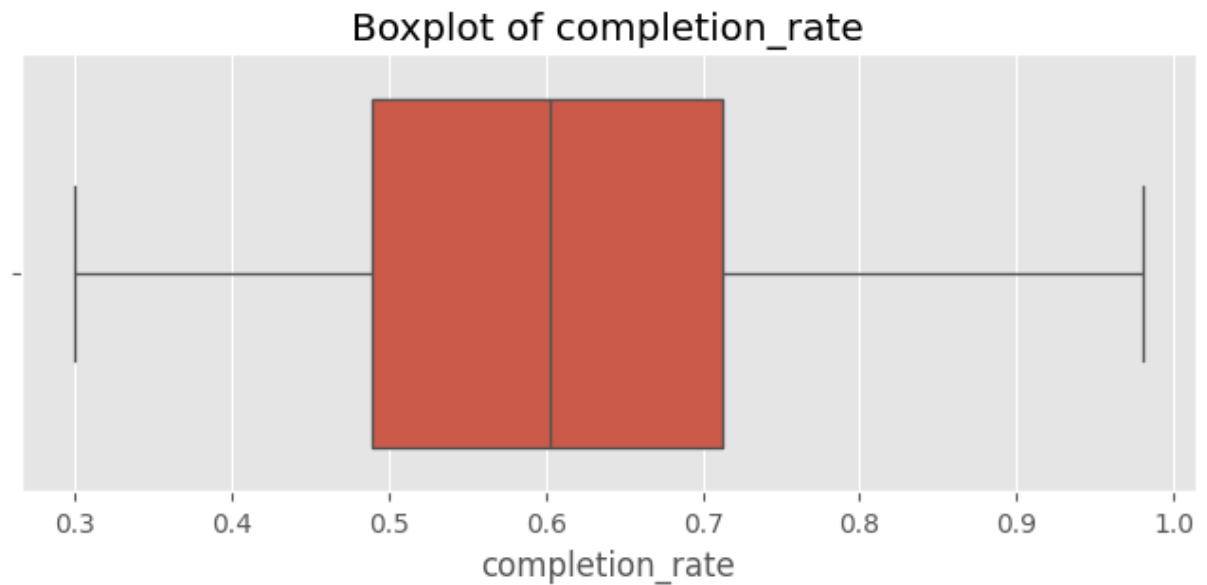


Distribution of avg_feedback_score

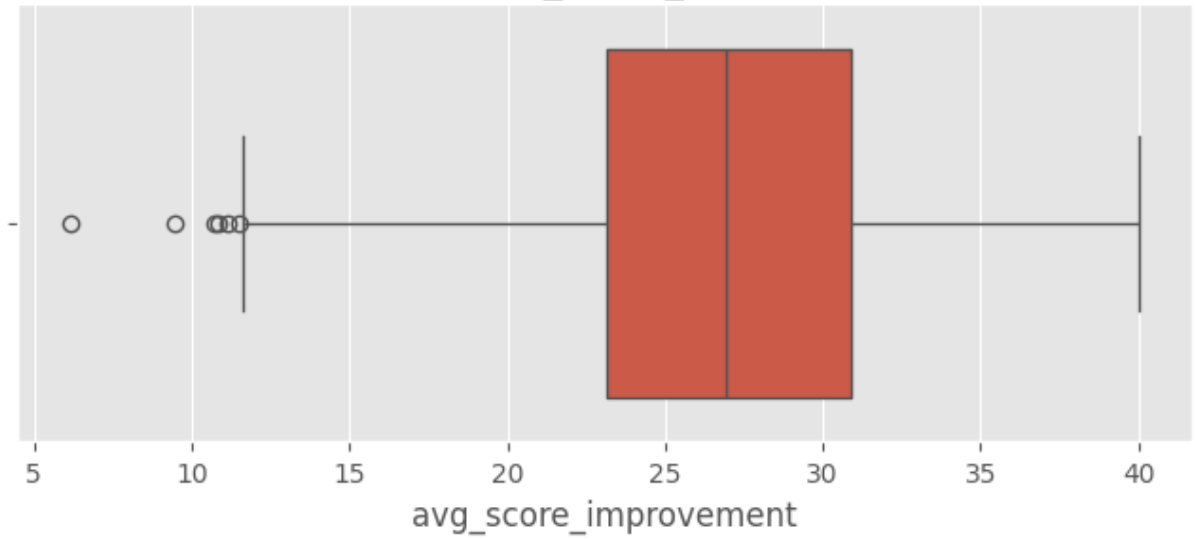




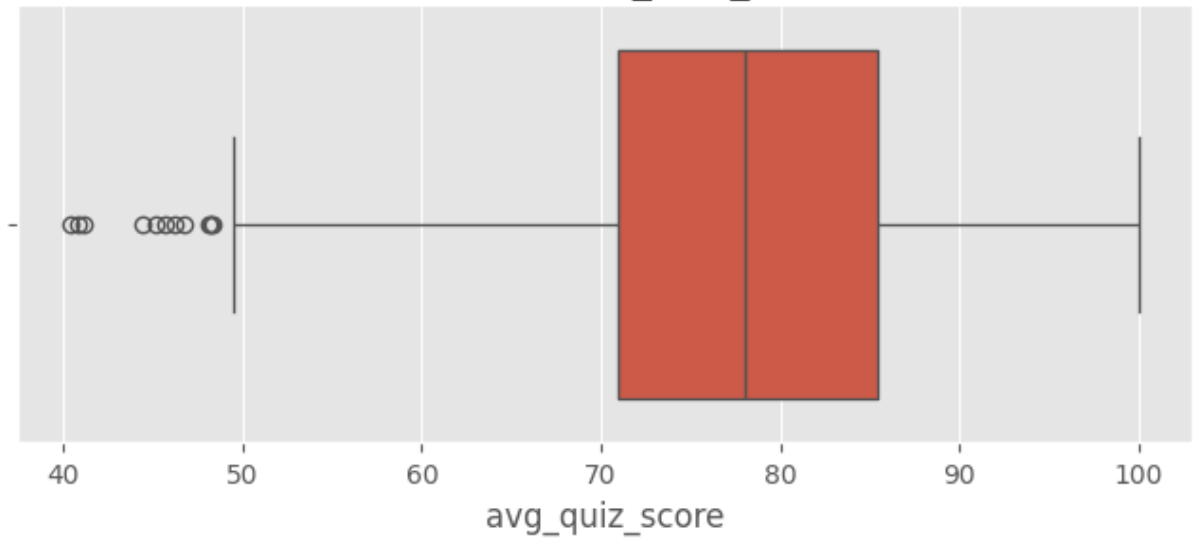
```
In [83]: for col in num_cols:
plt.figure(figsize=(8,3))
sns.boxplot(x=df[col])
plt.title(f"Boxplot of {col}")
plt.show()
```



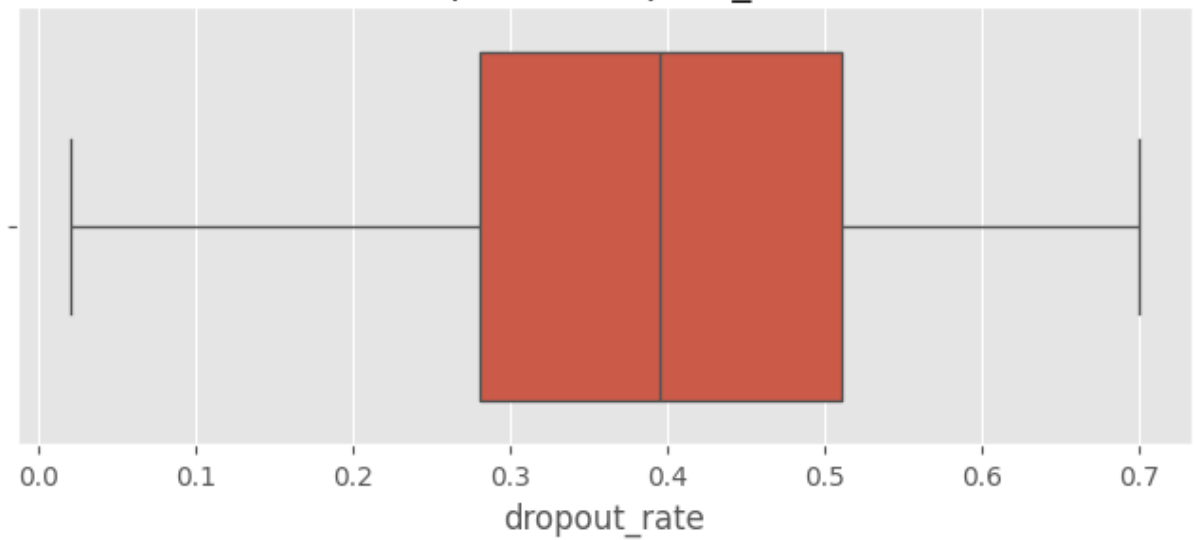
Boxplot of avg_score_improvement



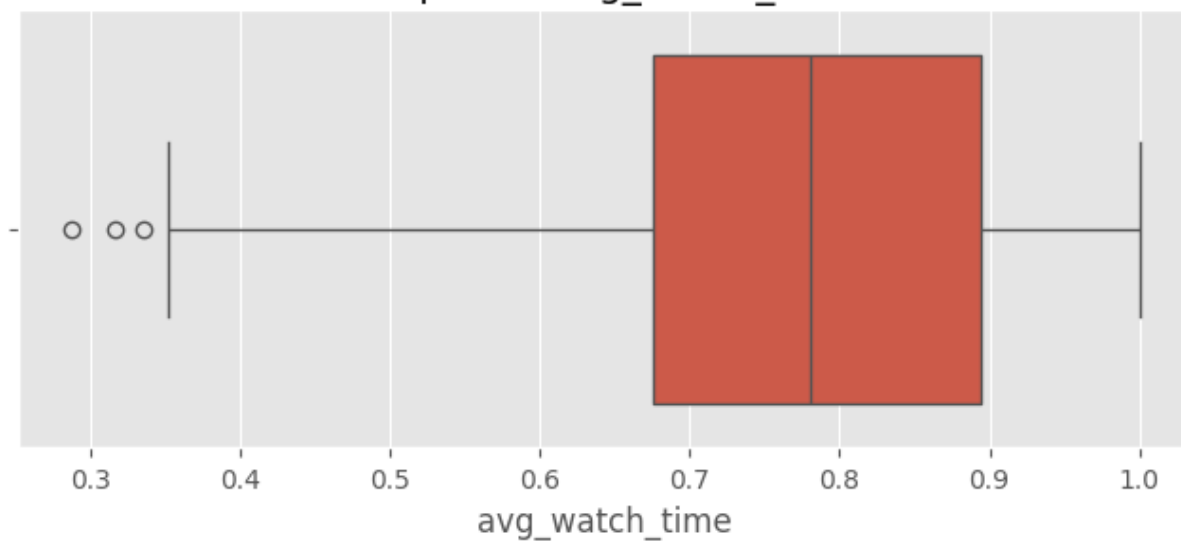
Boxplot of avg_quiz_score



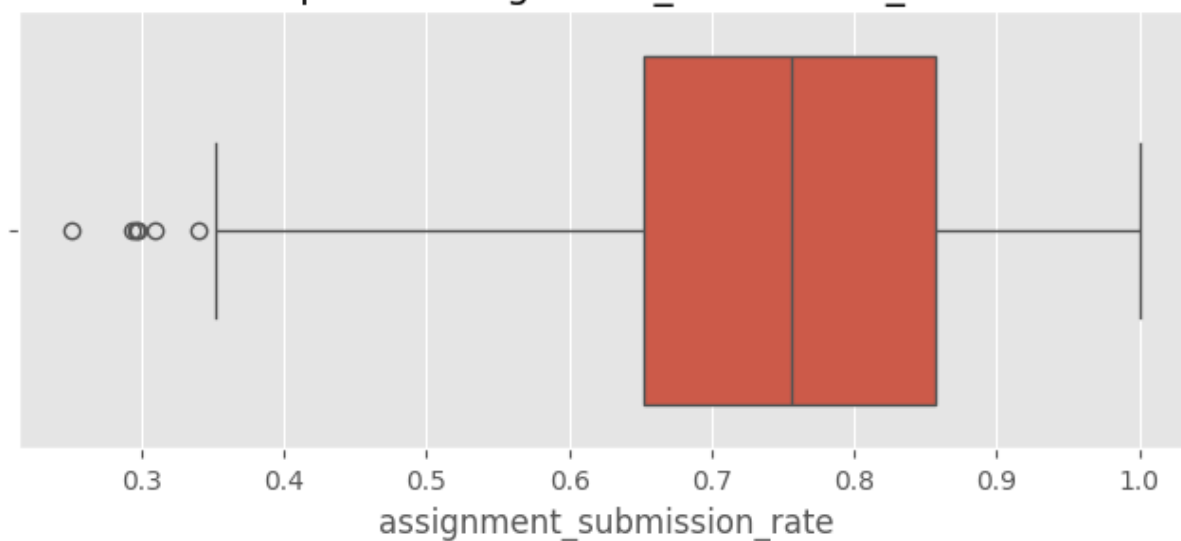
Boxplot of dropout_rate



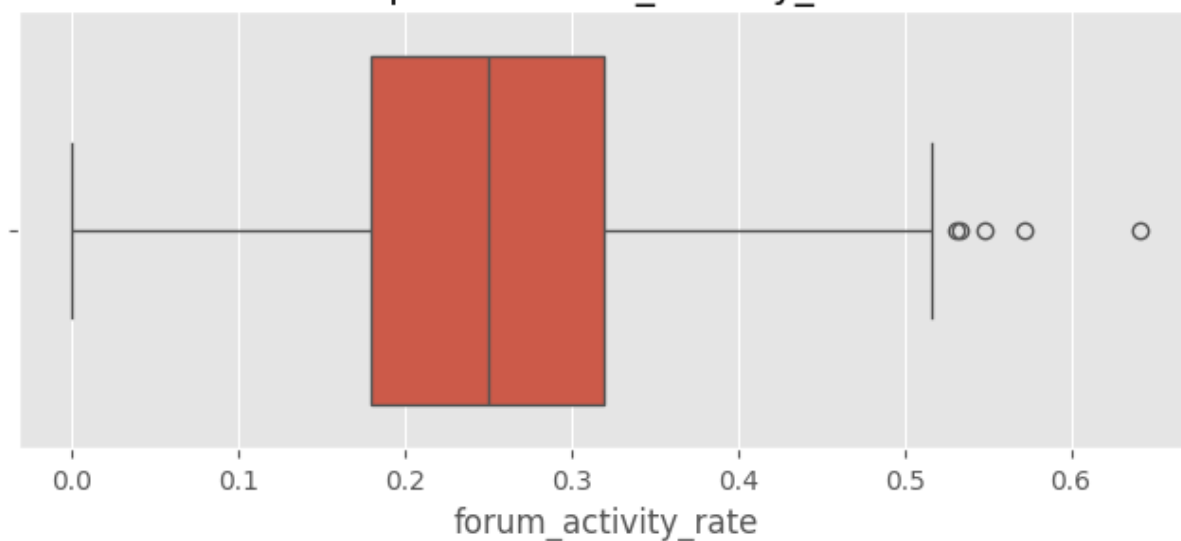
Boxplot of avg_watch_time

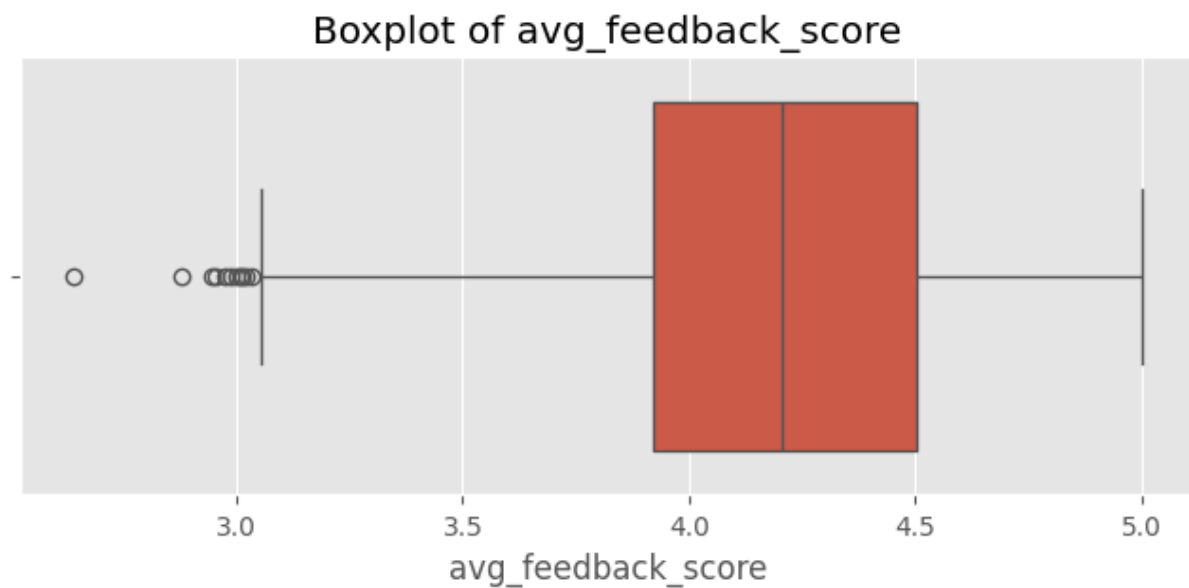


Boxplot of assignment_submission_rate



Boxplot of forum_activity_rate

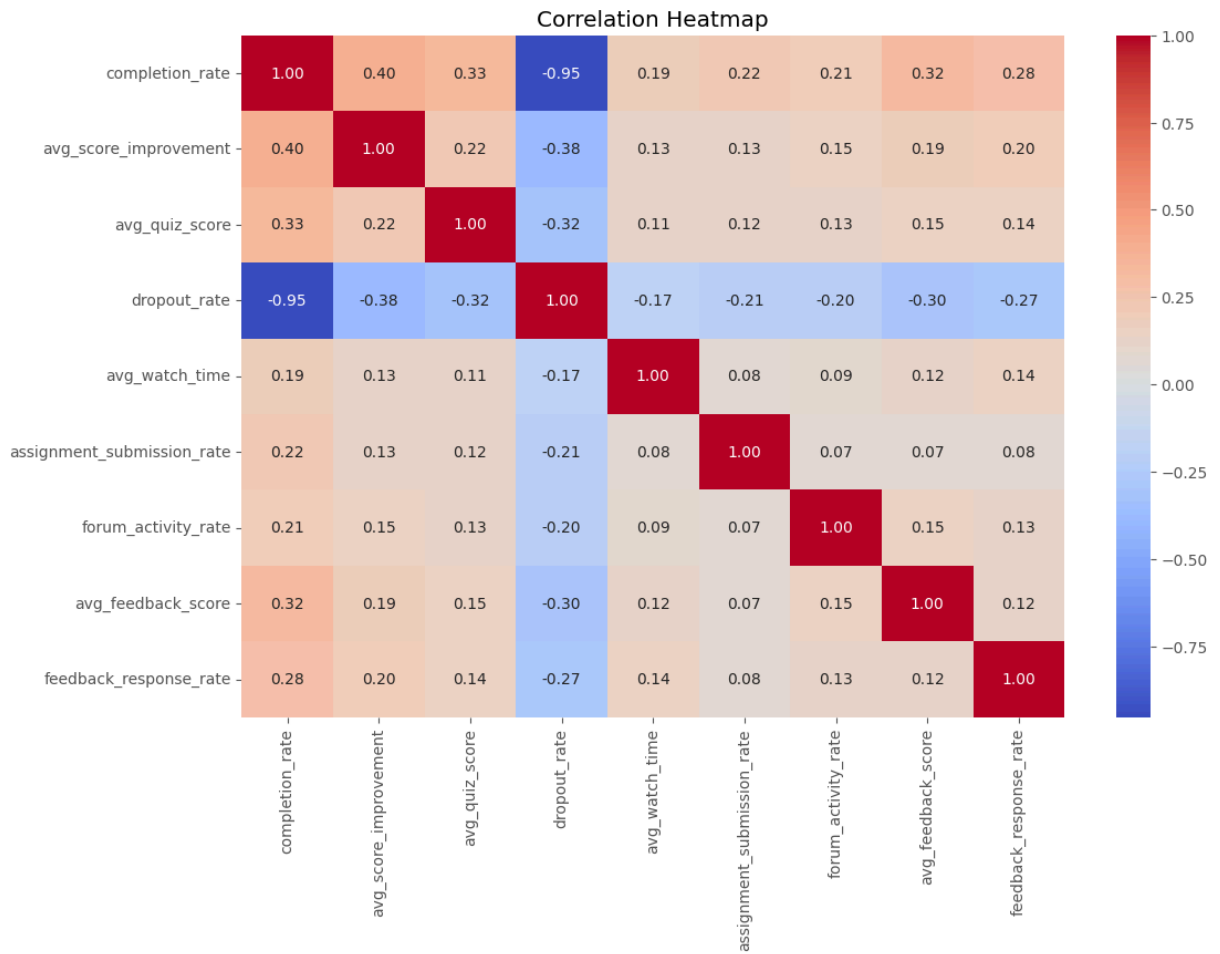




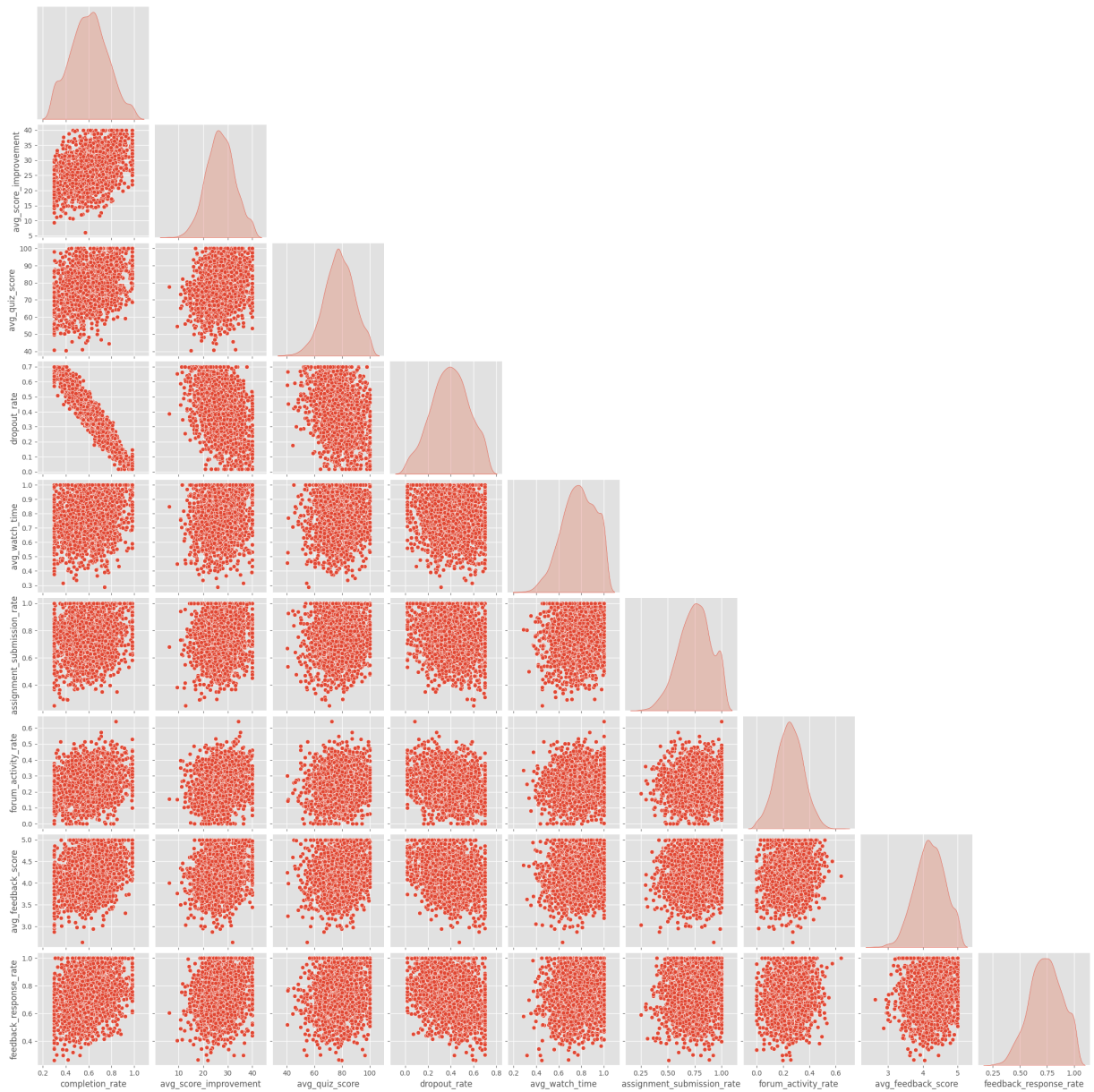
```
In [84]: corr=df[num_cols].corr()
plt.figure(figsize=(12,8))
sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Correlation Heatmap")
```



```
plt.show()
```



```
In [85]: sns.pairplot(  
df[num_cols],  
corner=True,  
diag_kind="kde"  
)  
plt.show()
```



```
In [86]: plt.figure(figsize=(8,5))

sns.histplot(

df["completion_rate"],

bins=30,

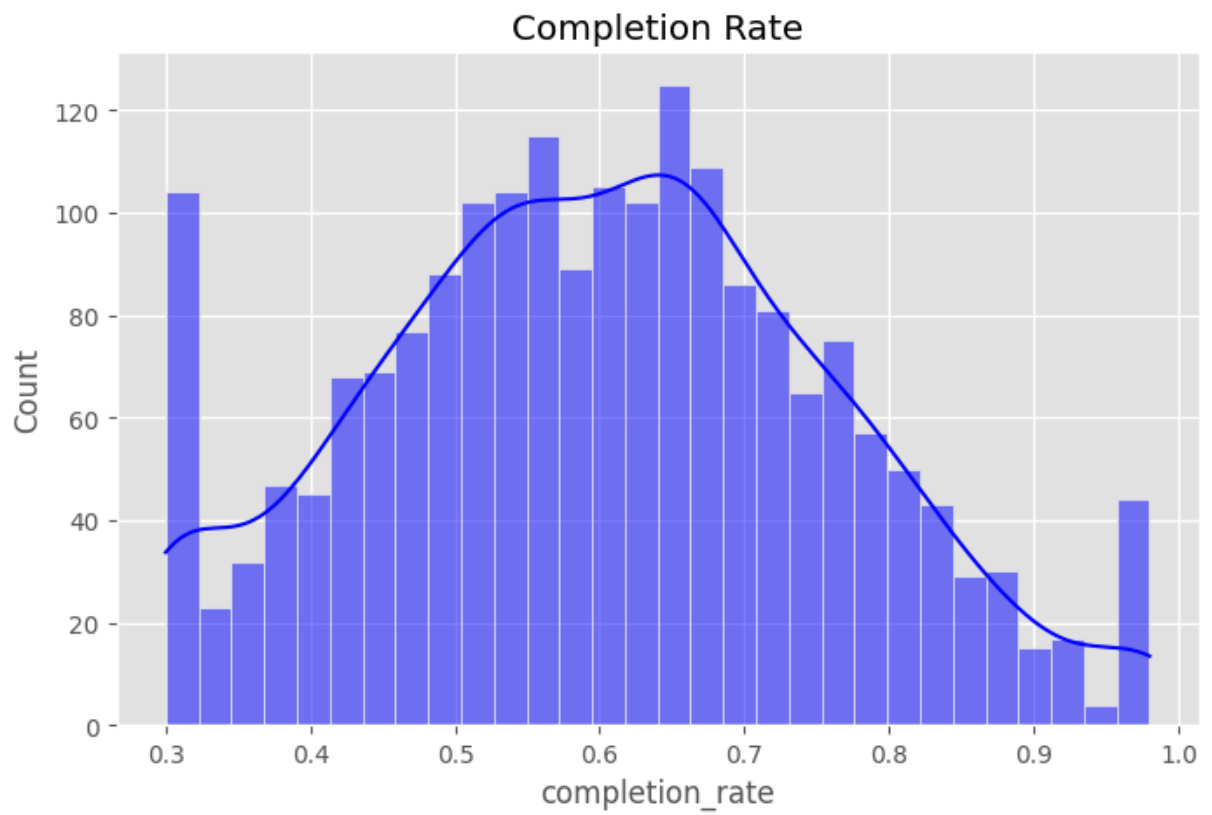
kde=True,

color="blue"

)

plt.title("Completion Rate")

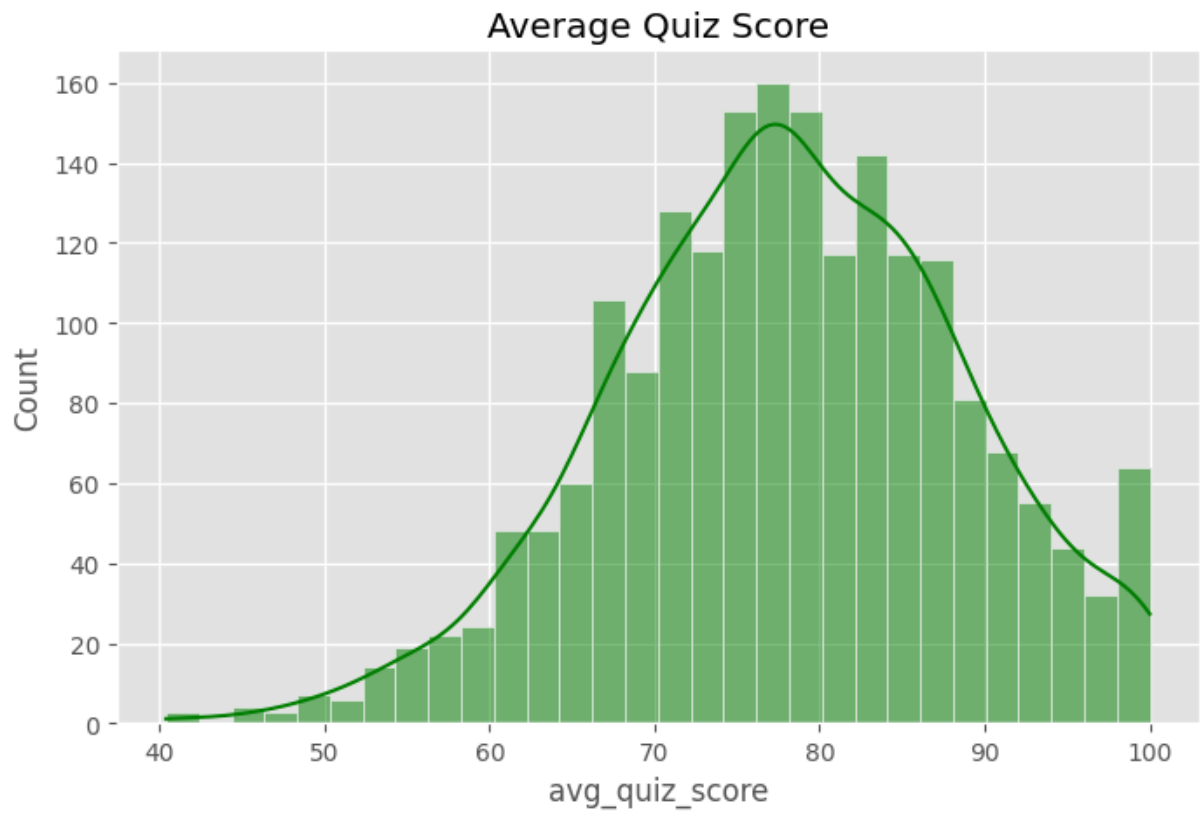
plt.show()
```



```
In [87]: plt.figure(figsize=(8,5))

sns.histplot(
    df["avg_quiz_score"],
    bins=30,
    kde=True,
    color="green"
)

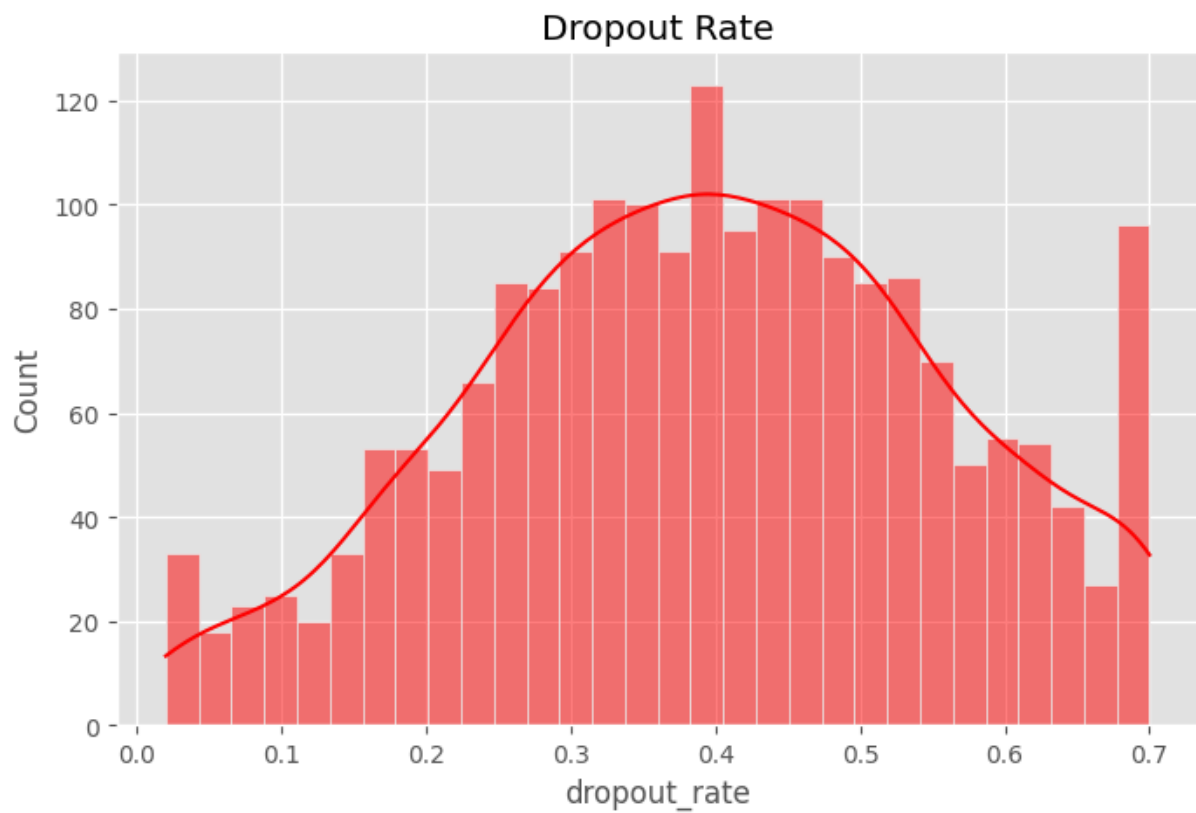
plt.title("Average Quiz Score")
plt.show()
```



```
In [88]: plt.figure(figsize=(8,5))

sns.histplot(
df["dropout_rate"],
bins=30,
kde=True,
color="red"
)

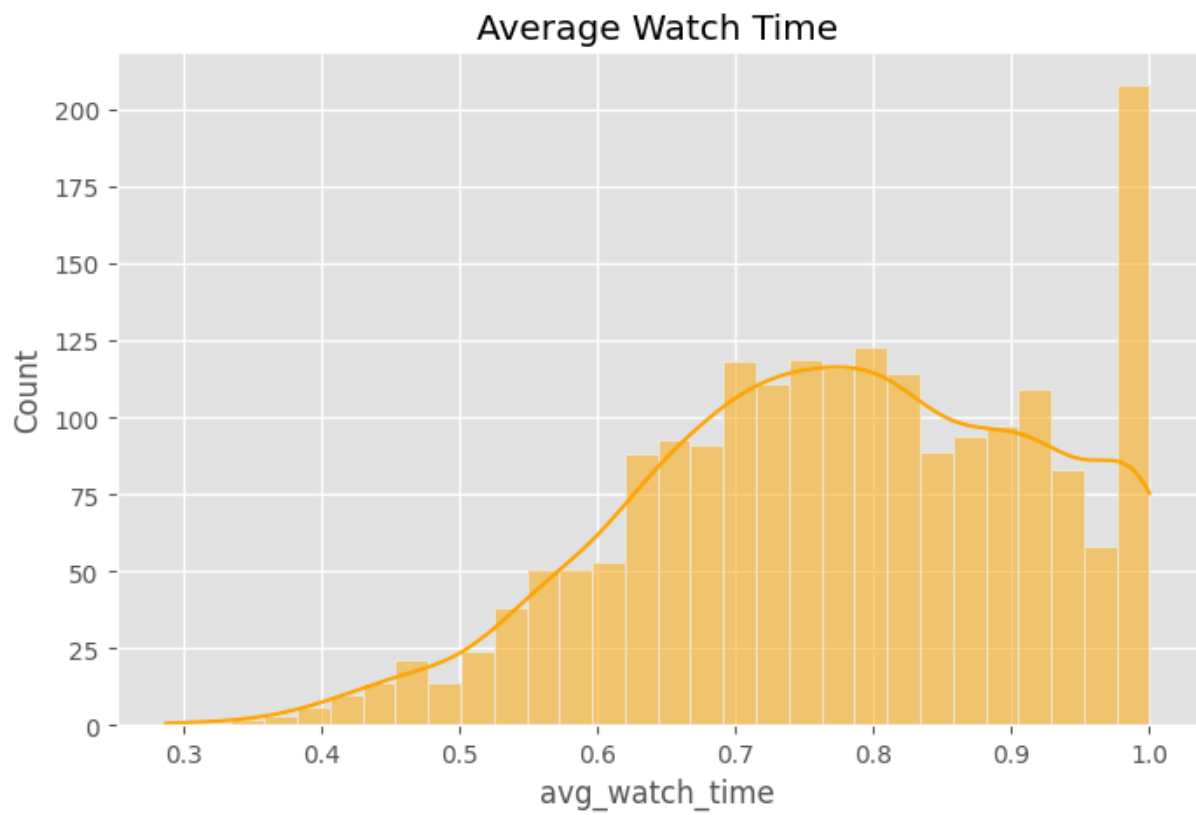
plt.title("Dropout Rate")
plt.show()
```



```
In [89]: plt.figure(figsize=(8,5))

sns.histplot(
    df["avg_watch_time"],
    bins=30,
    kde=True,
    color="orange"
)

plt.title("Average Watch Time")
plt.show()
```



```
In [90]: plt.figure(figsize=(8,5))

sns.histplot(

df["assignment_submission_rate"],

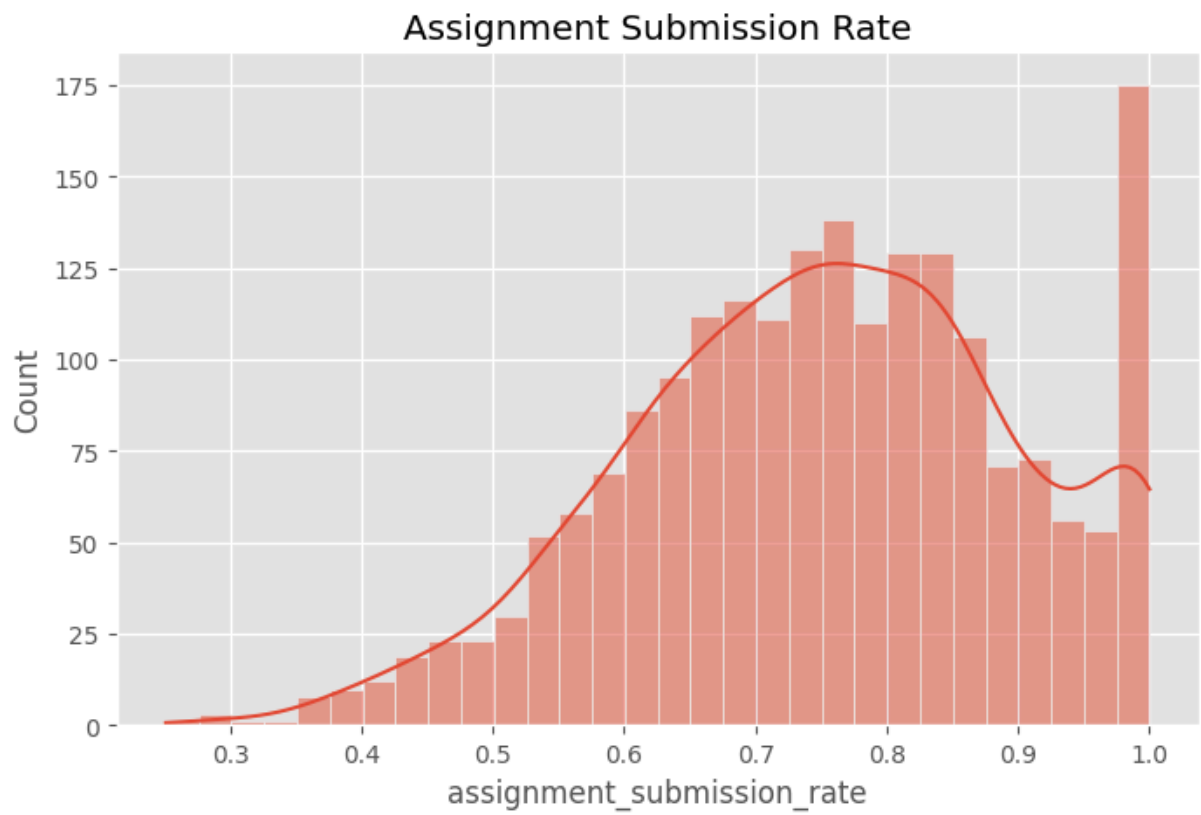
bins=30,

kde=True

)

plt.title("Assignment Submission Rate")

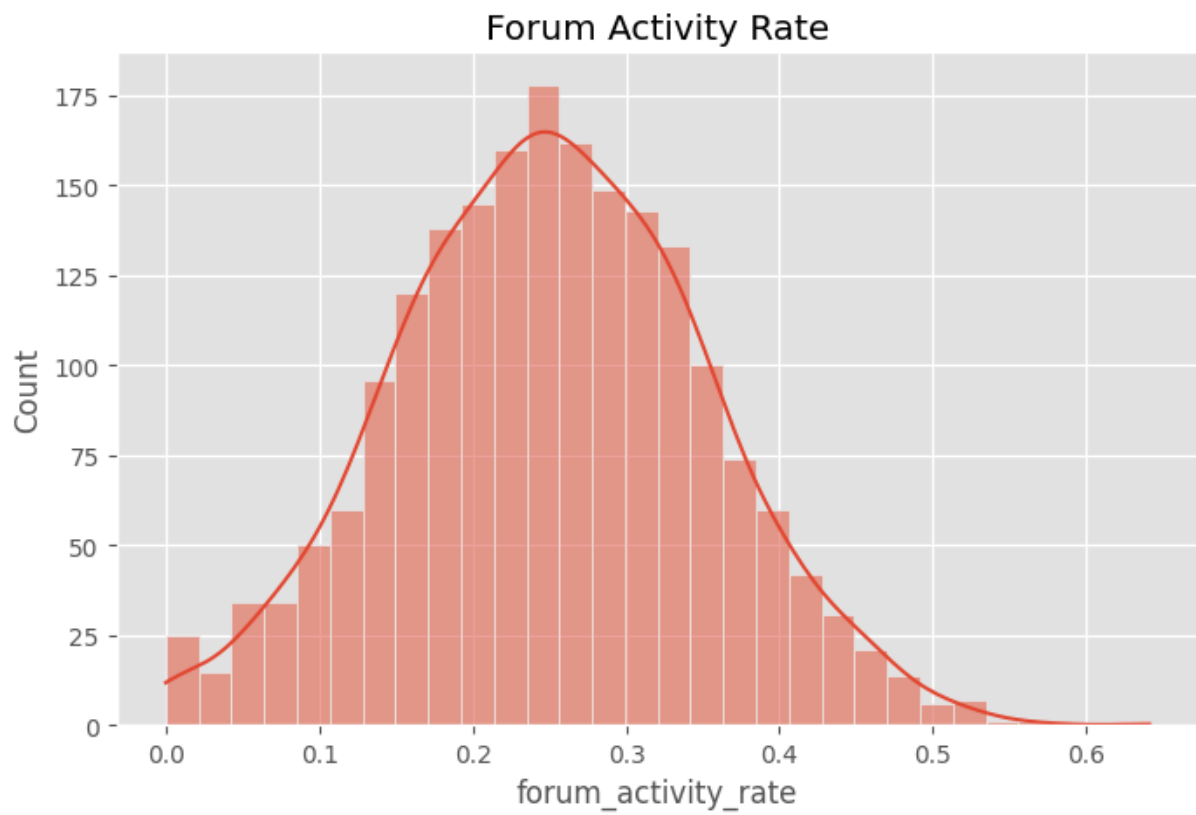
plt.show()
```



```
In [91]: plt.figure(figsize=(8,5))

sns.histplot(
    df["forum_activity_rate"],
    bins=30,
    kde=True
)

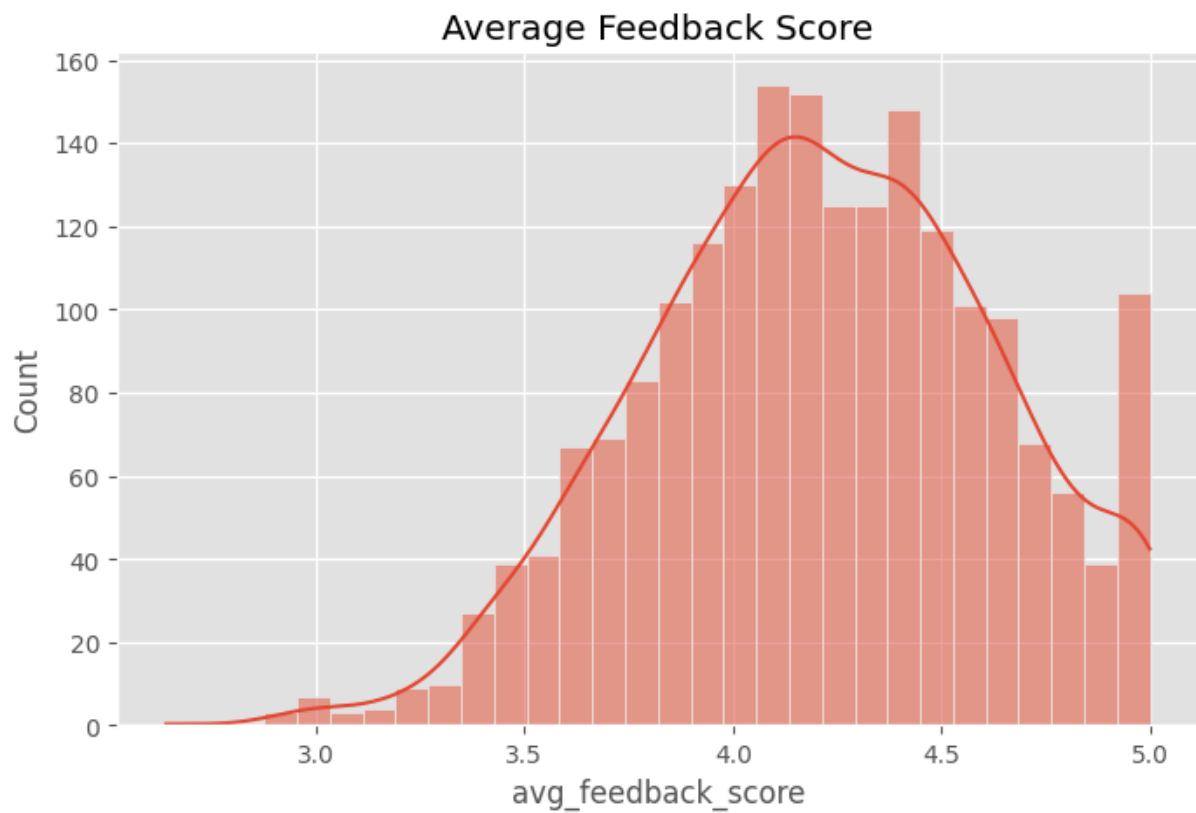
plt.title("Forum Activity Rate")
plt.show()
```



```
In [92]: plt.figure(figsize=(8,5))

sns.histplot(
    df["avg_feedback_score"],
    bins=30,
    kde=True
)

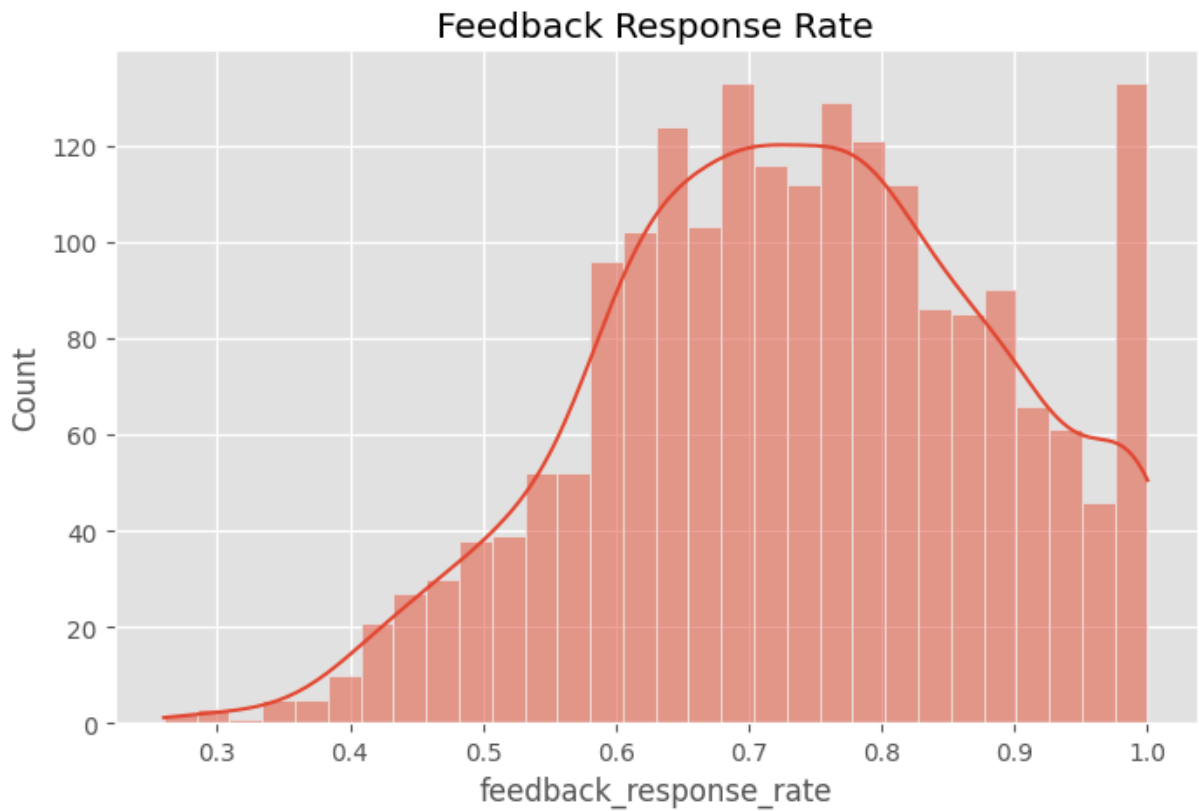
plt.title("Average Feedback Score")
plt.show()
```

```
In [93]: plt.figure(figsize=(8,5))

sns.histplot(
    df["feedback_response_rate"],
    bins=30,
    kde=True
)

plt.title("Feedback Response Rate")
plt.show()
```



```
In [94]: instructor=df.groupby("instructor_id").agg({
    "completion_rate":"mean",
    "avg_feedback_score":"mean",
    "avg_quiz_score":"mean"
})
instructor.head()
```

```
Out[94]:
```

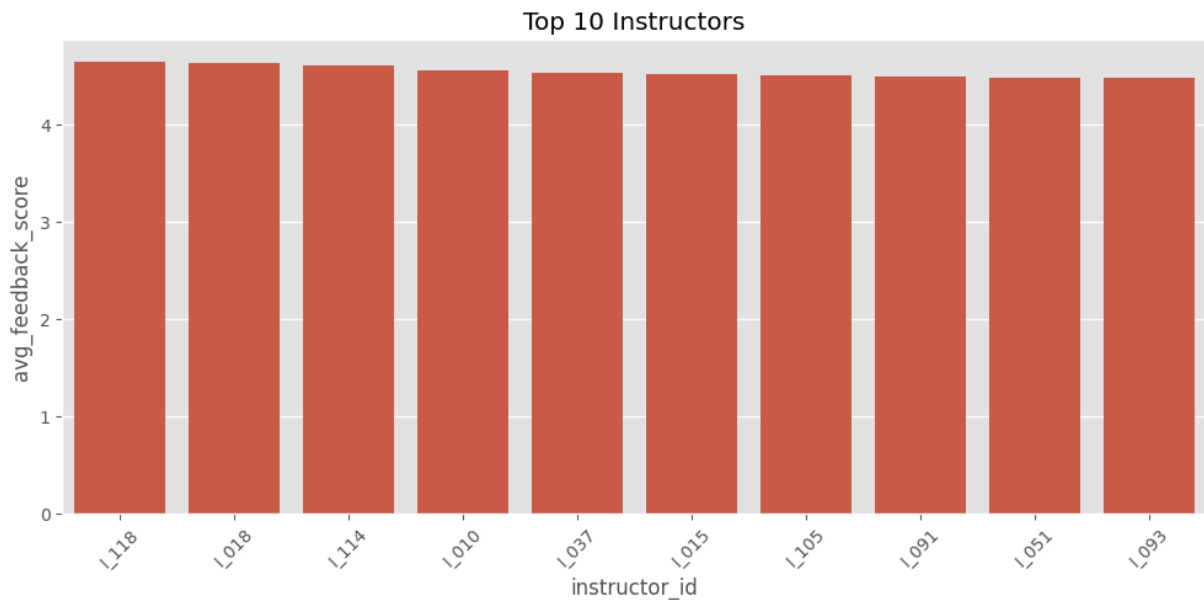
	completion_rate	avg_feedback_score	avg_quiz_score
instructor_id			
I_001	0.543887	4.217743	78.900194
I_002	0.730874	4.343002	81.737198
I_003	0.768447	4.450034	81.590928
I_004	0.458328	4.076410	77.680317
I_005	0.859747	4.202516	85.828159

```
In [95]: top10=instructor.sort_values(
    "avg_feedback_score",
```

```

ascending=False
).head(10)
plt.figure(figsize=(12,5))
sns.barplot(
x=top10.index,
y=top10["avg_feedback_score"]
)
plt.xticks(rotation=45)
plt.title("Top 10 Instructors")
plt.show()

```



```

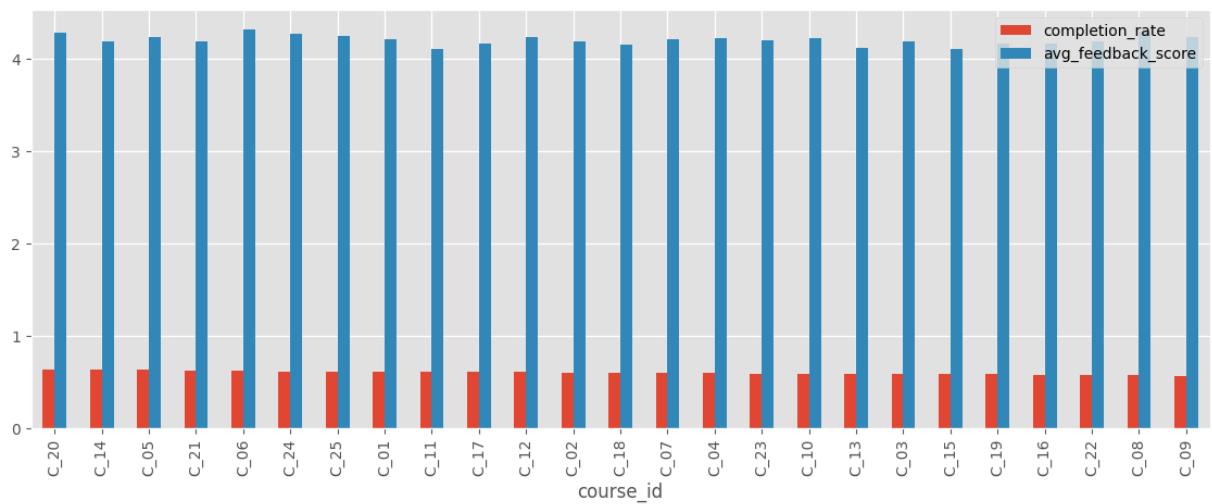
In [96]: course=df.groupby("course_id").agg({
    "completion_rate":"mean",
    "avg_feedback_score":"mean"
})
course.head()

```

Out[96]:

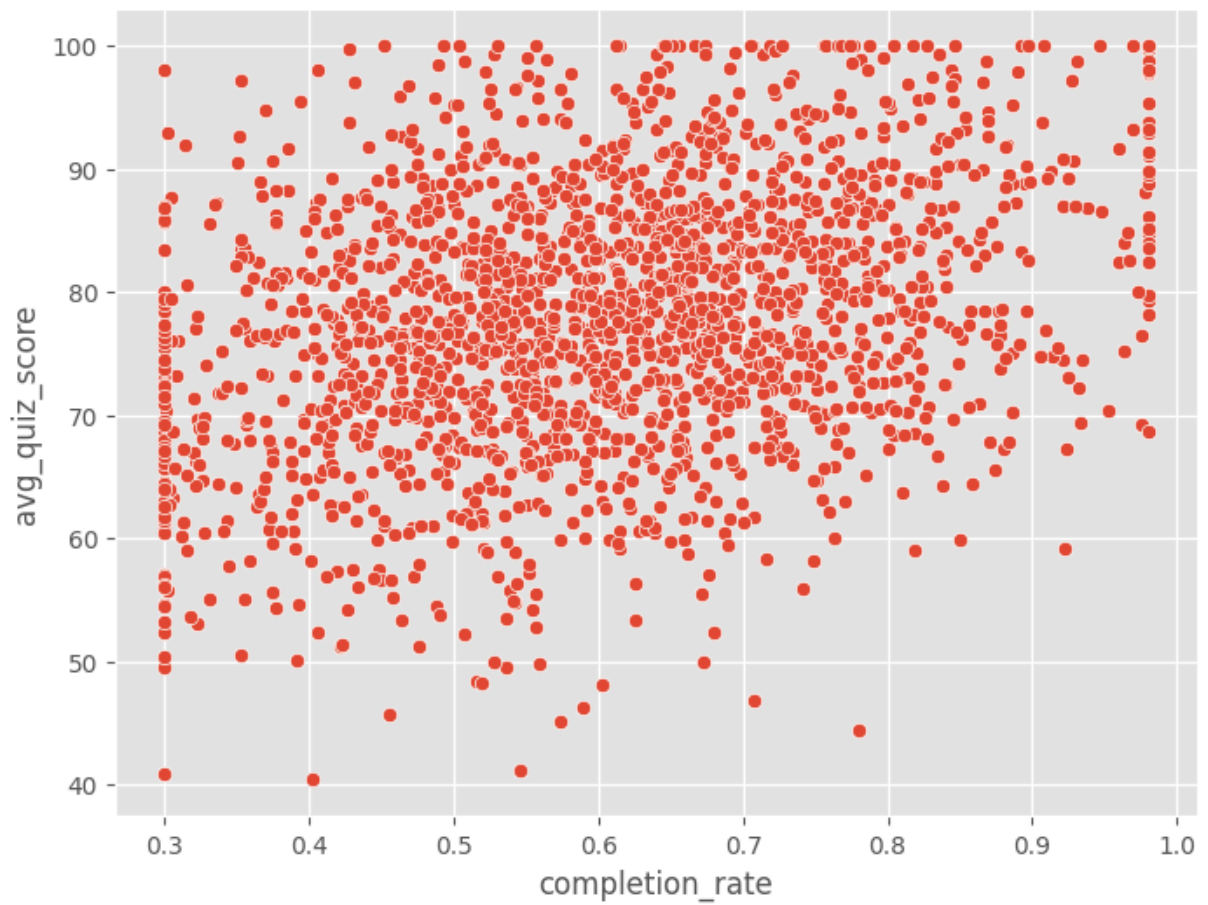
	completion_rate	avg_feedback_score
course_id		
C_01	0.611400	4.218196
C_02	0.601831	4.191351
C_03	0.592443	4.188071
C_04	0.597575	4.225682
C_05	0.630338	4.237671

```
In [97]: course.sort_values(  
    "completion_rate",  
    ascending=False  
).plot(  
    kind="bar",  
    figsize=(14,5)  
)  
plt.show()
```

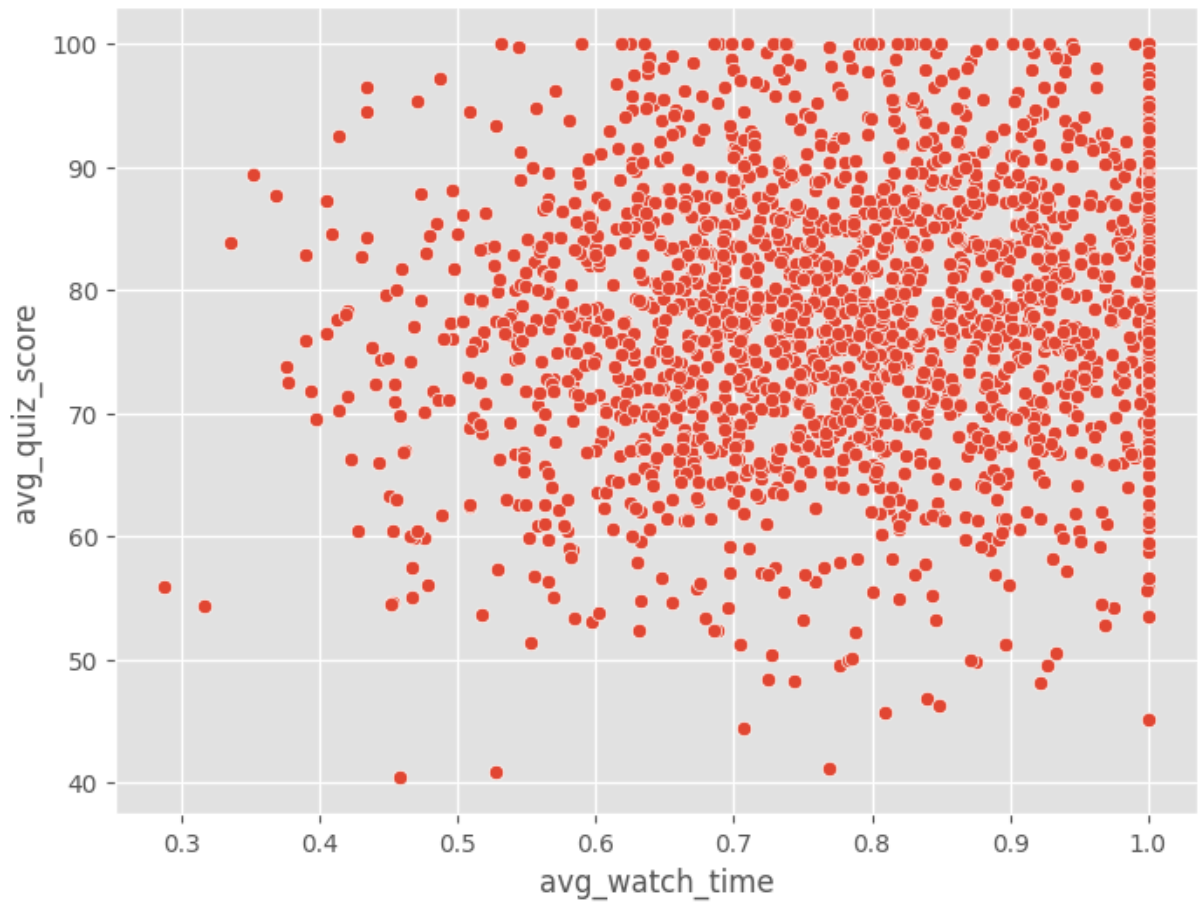


```
In [98]: plt.figure(figsize=(8,6))  
sns.scatterplot(  
    data=df,  
    x="completion_rate",  
    y="avg_quiz_score"
```

```
)  
plt.show()
```



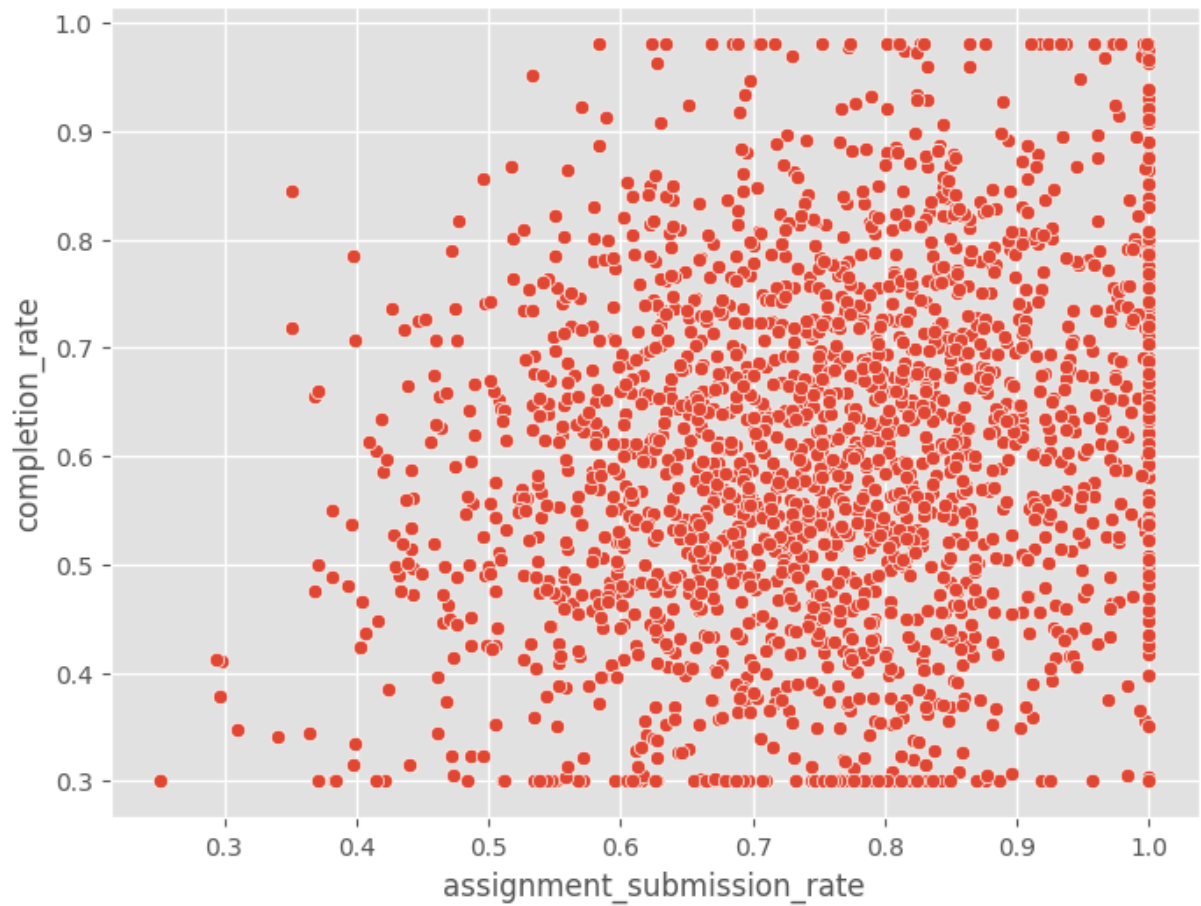
```
In [99]: plt.figure(figsize=(8,6))  
  
sns.scatterplot(  
    data=df,  
    x="avg_watch_time",  
    y="avg_quiz_score"  
)  
  
plt.show()
```



```
In [100... plt.figure(figsize=(8,6))

sns.scatterplot(
    data=df,
    x="assignment_submission_rate",
    y="completion_rate"
)

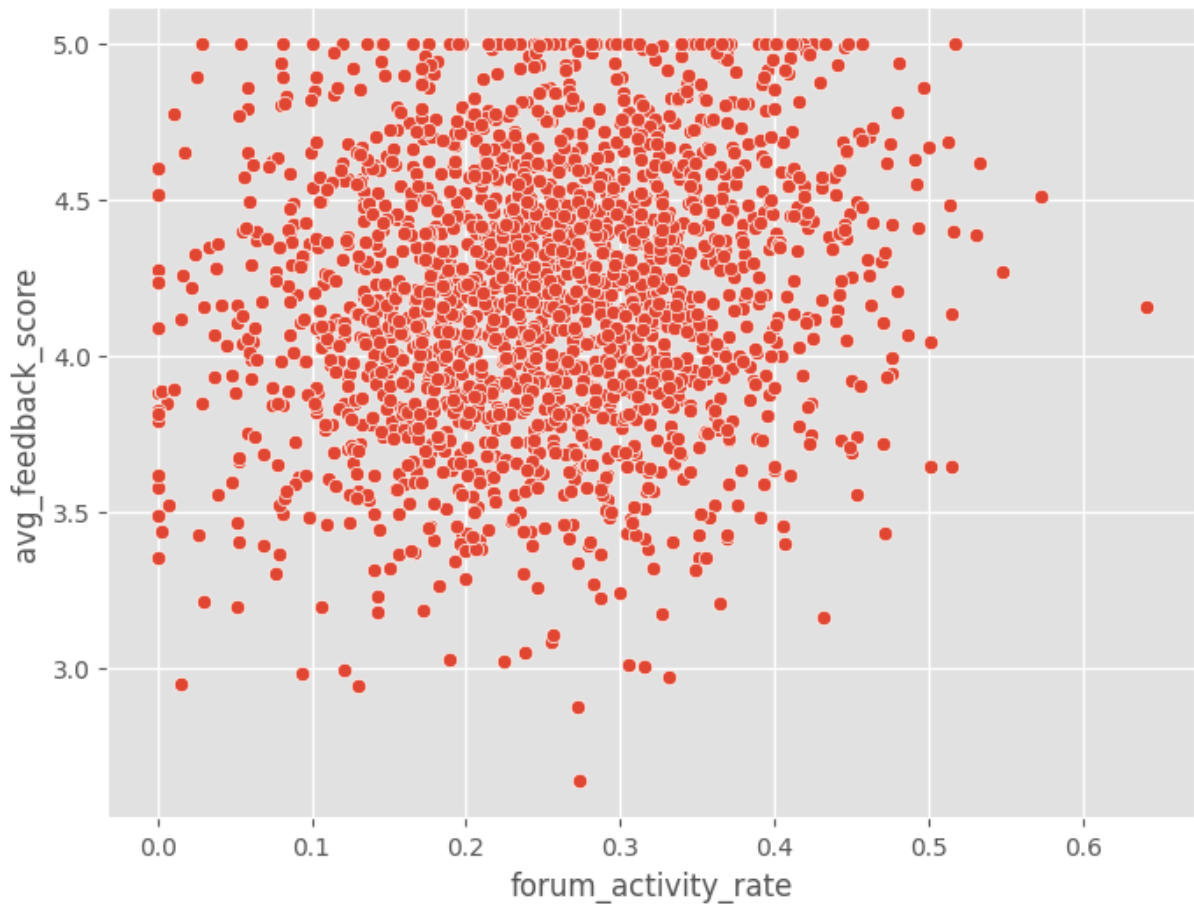
plt.show()
```



```
In [101... plt.figure(figsize=(8,6))

sns.scatterplot(
    data=df,
    x="forum_activity_rate",
    y="avg_feedback_score"
)

plt.show()
```



```
In [102... # Mean  
df[num_cols].mean()
```

```
Out[102... completion_rate      0.602808  
avg_score_improvement      27.035844  
avg_quiz_score             77.956126  
dropout_rate               0.394883  
avg_watch_time             0.776515  
assignment_submission_rate  0.753188  
forum_activity_rate        0.250300  
avg_feedback_score         4.207134  
feedback_response_rate     0.736519  
dtype: float64
```

```
In [103... # Median  
df[num_cols].median()
```

```
Out[103... completion_rate      0.603091  
avg_score_improvement      26.938629  
avg_quiz_score             78.020567  
dropout_rate               0.394820  
avg_watch_time             0.780330  
assignment_submission_rate  0.756380  
forum_activity_rate        0.249771  
avg_feedback_score         4.205989  
feedback_response_rate     0.737213  
dtype: float64
```



```
In [104... # Mode
df[num_cols].mode().iloc[0]
```

```
Out[104... completion_rate      0.3
avg_score_improvement    40.0
avg_quiz_score          100.0
dropout_rate             0.7
avg_watch_time           1.0
assignment_submission_rate 1.0
forum_activity_rate      0.0
avg_feedback_score       5.0
feedback_response_rate   1.0
Name: 0, dtype: float64
```

```
In [105... df[num_cols].std()
```

```
Out[105... completion_rate      0.159667
avg_score_improvement    5.716641
avg_quiz_score          10.695618
dropout_rate             0.162747
avg_watch_time           0.145231
assignment_submission_rate 0.148058
forum_activity_rate      0.100640
avg_feedback_score       0.419209
feedback_response_rate   0.149412
dtype: float64
```

```
In [106... df[num_cols].var()
```

```
Out[106... completion_rate      0.025493
avg_score_improvement    32.679979
avg_quiz_score          114.396245
dropout_rate             0.026486
avg_watch_time           0.021092
assignment_submission_rate 0.021921
forum_activity_rate      0.010128
avg_feedback_score       0.175736
feedback_response_rate   0.022324
dtype: float64
```

```
In [107... df[num_cols].skew()
```

```
Out[107... completion_rate      0.109298
avg_score_improvement    -0.003487
avg_quiz_score          -0.200492
dropout_rate            -0.102811
avg_watch_time          -0.317859
assignment_submission_rate -0.255980
forum_activity_rate      0.044511
avg_feedback_score       -0.172339
feedback_response_rate   -0.175486
dtype: float64
```

```
In [108... df[num_cols].kurt()
```

```
Out[108... completion_rate      -0.450145
avg_score_improvement    -0.165707
avg_quiz_score           -0.066024
dropout_rate             -0.568822
avg_watch_time           -0.492592
assignment_submission_rate -0.373800
forum_activity_rate       -0.100724
avg_feedback_score        -0.298692
feedback_response_rate    -0.438173
dtype: float64
```

```
In [109... cov_matrix = df[num_cols].cov()

cov_matrix
```

```
Out[109...      completion_rate  avg_score_improvement  avg_quiz_score
completion_rate      0.025493             0.368901         0.569200
avg_score_improvement 0.368901          32.679979        13.377326
avg_quiz_score        0.569200          13.377326        114.396245
dropout_rate          -0.024777          -0.354798         -0.548696
avg_watch_time        0.004351           0.106562         0.171873
assignment_submission_rate 0.005245         0.109112         0.183185
forum_activity_rate    0.003307           0.087819         0.138420
avg_feedback_score     0.021110           0.455459         0.671771
feedback_response_rate 0.006669           0.169907         0.231371
```

```
In [110... corr = df[num_cols].corr()

corr_pairs = corr.unstack().sort_values(ascending=False)

corr_pairs.drop_duplicates().head(20)
```

```

Out[110... completion_rate      completion_rate      1.000000
avg_score_improvement  completion_rate      0.404161
avg_quiz_score         completion_rate      0.333307
completion_rate        avg_feedback_score    0.315394
feedback_response_rate completion_rate      0.279549
assignment_submission_rate completion_rate    0.221877
avg_score_improvement  avg_quiz_score      0.218788
forum_activity_rate    completion_rate      0.205793
avg_score_improvement  feedback_response_rate 0.198923
                        avg_feedback_score    0.190054
avg_watch_time         completion_rate      0.187633
avg_score_improvement  forum_activity_rate  0.152643
avg_quiz_score         avg_feedback_score    0.149825
avg_feedback_score     forum_activity_rate  0.147115
feedback_response_rate avg_quiz_score        0.144783
avg_watch_time         feedback_response_rate 0.138435
assignment_submission_rate avg_score_improvement 0.128914
avg_quiz_score         forum_activity_rate  0.128595
avg_score_improvement  avg_watch_time        0.128352
forum_activity_rate    feedback_response_rate 0.126143
dtype: float64

```

```

In [111... instructor_rank = (

    df.groupby("instructor_id")

    .agg({

        "completion_rate": "mean",

        "avg_quiz_score": "mean",

        "avg_feedback_score": "mean",

        "dropout_rate": "mean"

    })

)

instructor_rank

```

Out[111...

	completion_rate	avg_quiz_score	avg_feedback_score	dropout_rate
instructor_id				
I_001	0.543887	78.900194	4.217743	0.470593
I_002	0.730874	81.737198	4.343002	0.247194
I_003	0.768447	81.590928	4.450034	0.234828
I_004	0.458328	77.680317	4.076410	0.547261
I_005	0.859747	85.828159	4.202516	0.145733
...	...	...	...	...
I_116	0.597106	78.640236	4.091469	0.412782
I_117	0.691715	80.981607	4.305839	0.312481
I_118	0.820753	87.071798	4.641986	0.202034
I_119	0.600597	80.735733	4.315403	0.399761
I_120	0.570574	75.408861	4.126397	0.427318

120 rows × 4 columns

In [112...

```
df["performance_score"]=(
df["completion_rate"]*0.30+
(df["avg_quiz_score"]/100)*0.30+
(df["avg_feedback_score"]/5)*0.20+
(1-df["dropout_rate"])*0.20
)
```

In [113...

```
top=df.groupby("instructor_id")["performance_score"]\
.mean()\
.sort_values(ascending=False)\
.head(15)

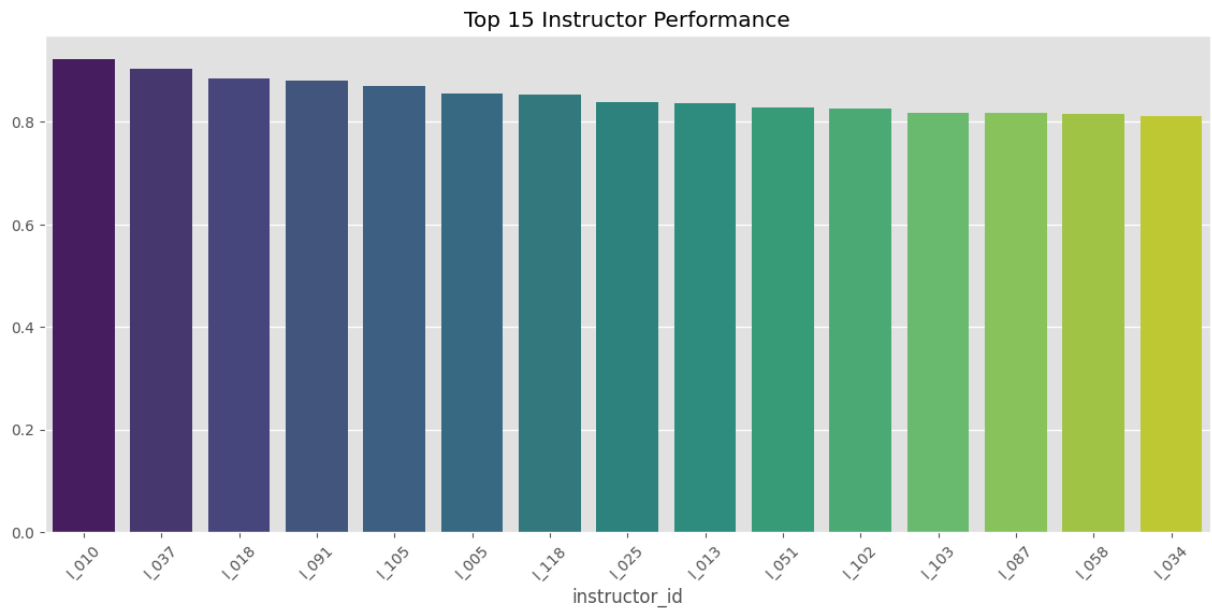
plt.figure(figsize=(14,6))

sns.barplot(
x=top.index,
y=top.values,
palette="viridis"
)
```

```
plt.xticks(rotation=45)

plt.title("Top 15 Instructor Performance")

plt.show()
```



```
In [114... bottom=df.groupby("instructor_id")["performance_score"]\
            .mean()\
            .sort_values()\
            .head(15)

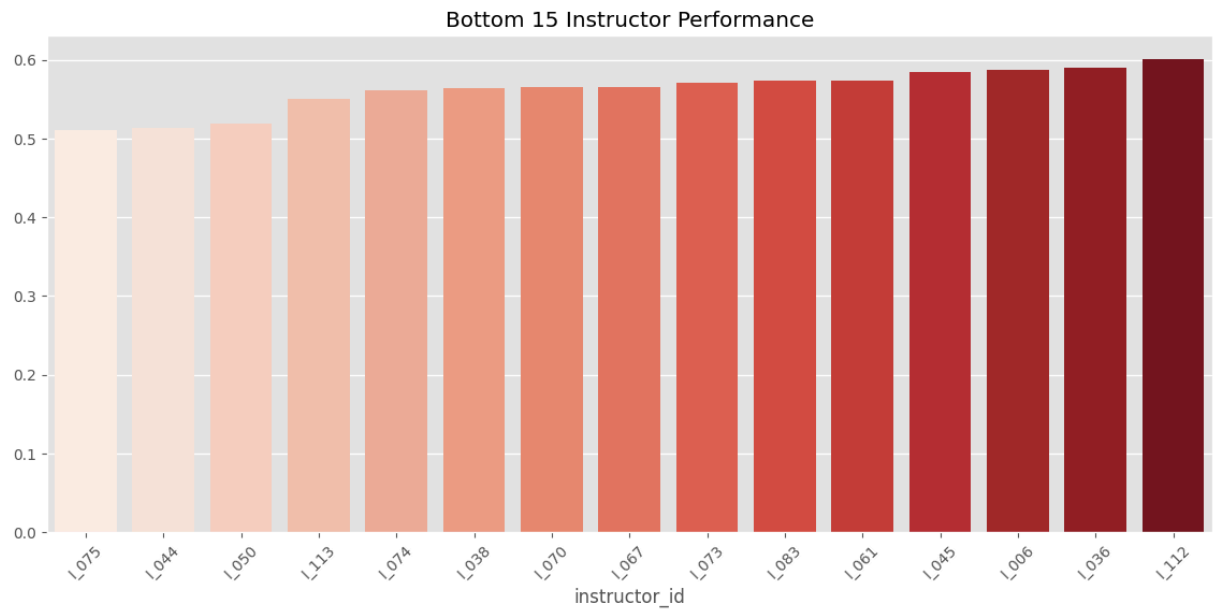
plt.figure(figsize=(14,6))

sns.barplot(
    x=bottom.index,
    y=bottom.values,
    palette="Reds"
)

plt.xticks(rotation=45)

plt.title("Bottom 15 Instructor Performance")

plt.show()
```



```
In [115... from sklearn.preprocessing import StandardScaler  
  
scaler=StandardScaler()  
  
X=scaler.fit_transform(df[num_cols])
```

```
In [116... from sklearn.decomposition import PCA  
  
pca=PCA()  
  
X_pca=pca.fit_transform(X)
```

```
In [117... variance=pd.DataFrame({  
  
    "Principal Component":  
    range(1,len(pca.explained_variance_ratio_)+1),  
  
    "Explained Variance":  
    pca.explained_variance_ratio_  
})  
  
variance
```

Out[117...

	Principal Component	Explained Variance
0	1	0.324763
1	2	0.108104
2	3	0.104820
3	4	0.099994
4	5	0.096892
5	6	0.092444
6	7	0.085776
7	8	0.082118
8	9	0.005088

In [118...

```
variance["Cumulative"] = variance["Explained Variance"].cumsum()  
variance
```

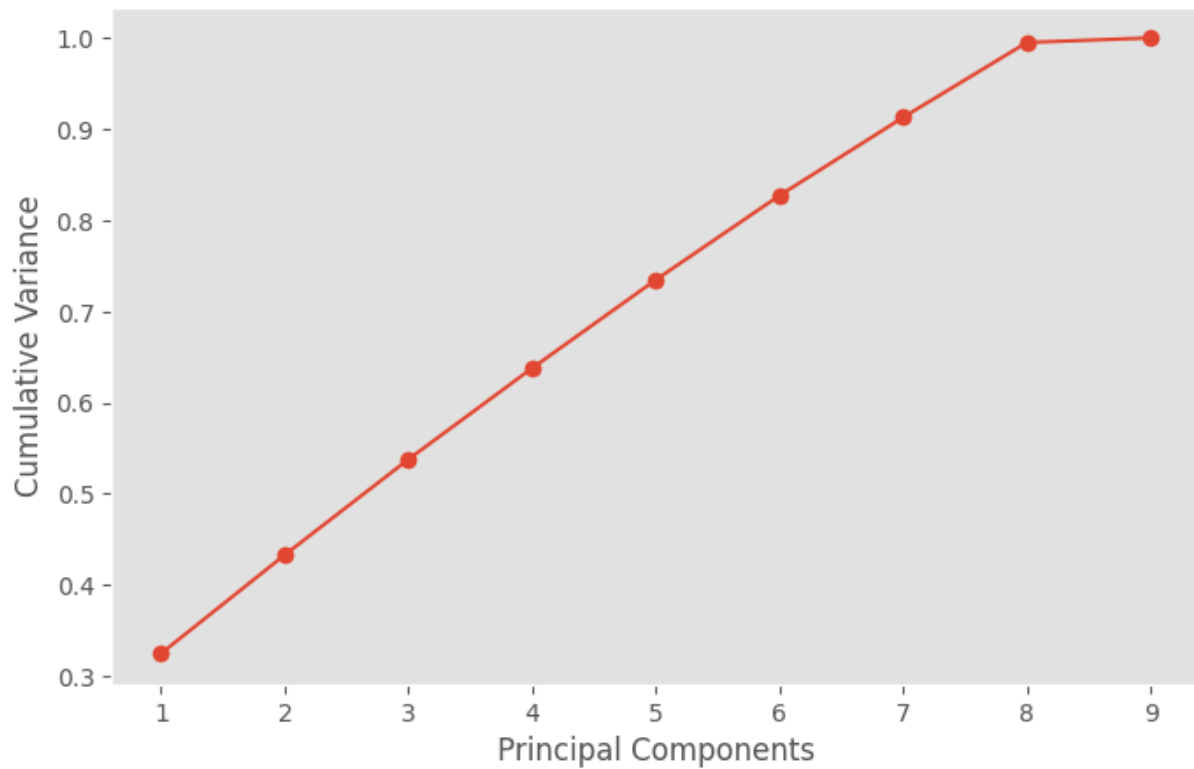
Out[118...

	Principal Component	Explained Variance	Cumulative
0	1	0.324763	0.324763
1	2	0.108104	0.432867
2	3	0.104820	0.537687
3	4	0.099994	0.637681
4	5	0.096892	0.734574
5	6	0.092444	0.827018
6	7	0.085776	0.912794
7	8	0.082118	0.994912
8	9	0.005088	1.000000

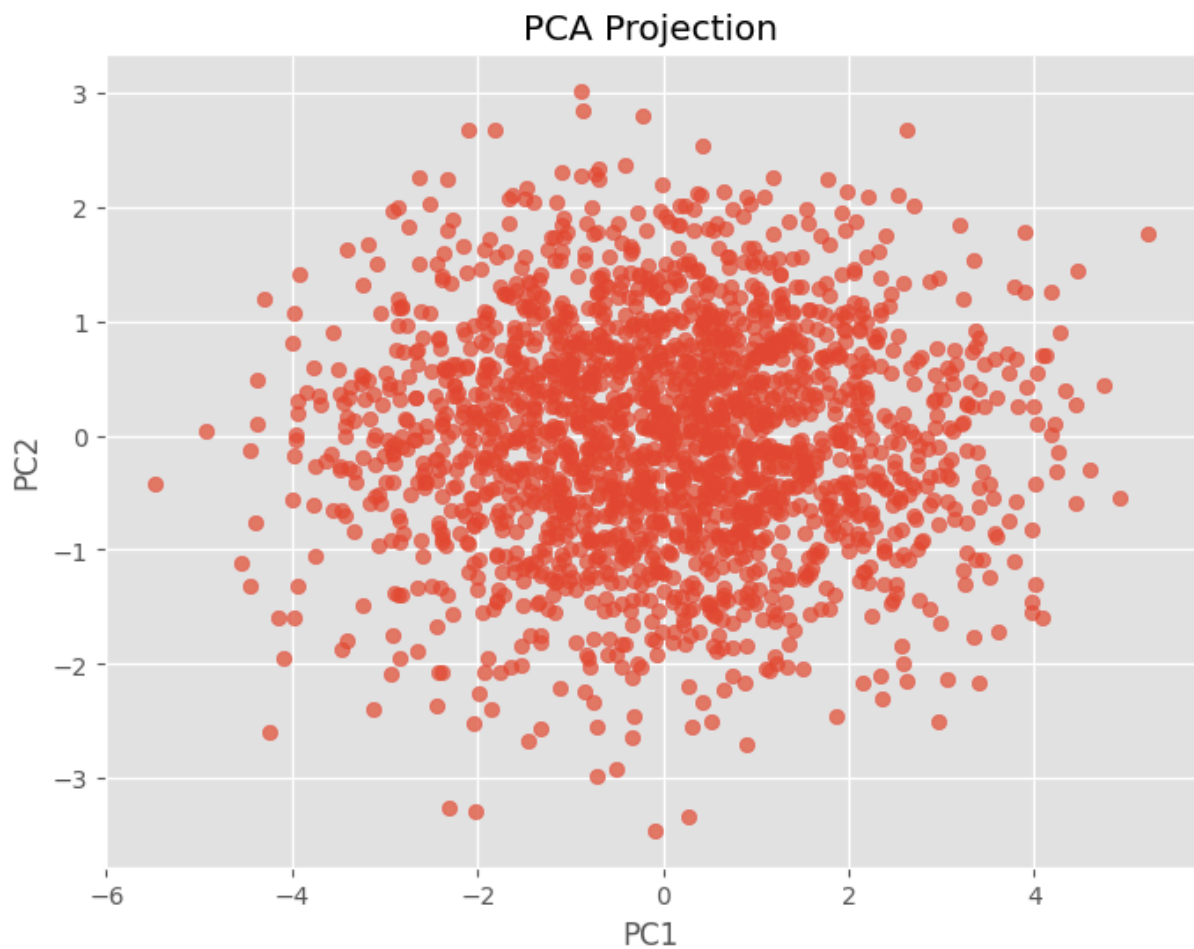
In [119...

```
plt.figure(figsize=(8,5))  
plt.plot(  
variance["Principal Component"],  
variance["Cumulative"],  
marker="o"  
)
```

```
plt.xlabel("Principal Components")
plt.ylabel("Cumulative Variance")
plt.grid()
plt.show()
```



```
In [120... plt.figure(figsize=(8,6))
plt.scatter(
X_pca[:,0],
X_pca[:,1],
alpha=0.7
)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Projection")
plt.show()
```

```
In [121...] from sklearn.cluster import KMeans
```

```
wcss=[]
```

```
for i in range(1,11):
```

```
    km=KMeans(
```

```
        n_clusters=i,
```

```
        random_state=42,
```

```
        n_init=10
```

```
    )
```

```
    km.fit(X)
```

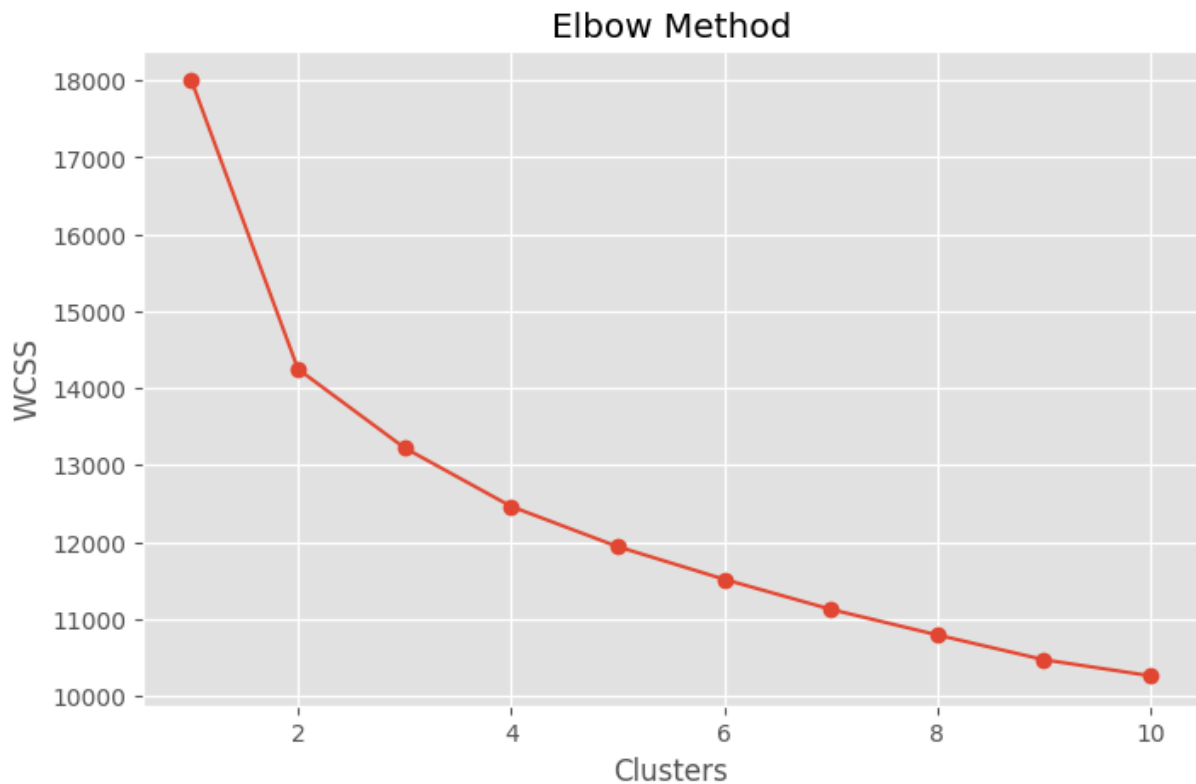
```
    wcss.append(km.inertia_)
```

```
In [122...] plt.figure(figsize=(8,5))
```

```
plt.plot(
```

```
    range(1,11),
```

```
wcss,
marker="o"
)
plt.xlabel("Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```



```
In [123... kmeans=KMeans(
n_clusters=3,
random_state=42,
n_init=10
)
labels=kmeans.fit_predict(X)
df["Cluster"]=labels
```

```
In [124... from sklearn.metrics import silhouette_score
score=silhouette_score(X,labels)
```

```
print("Silhouette Score :",round(score,3))
```

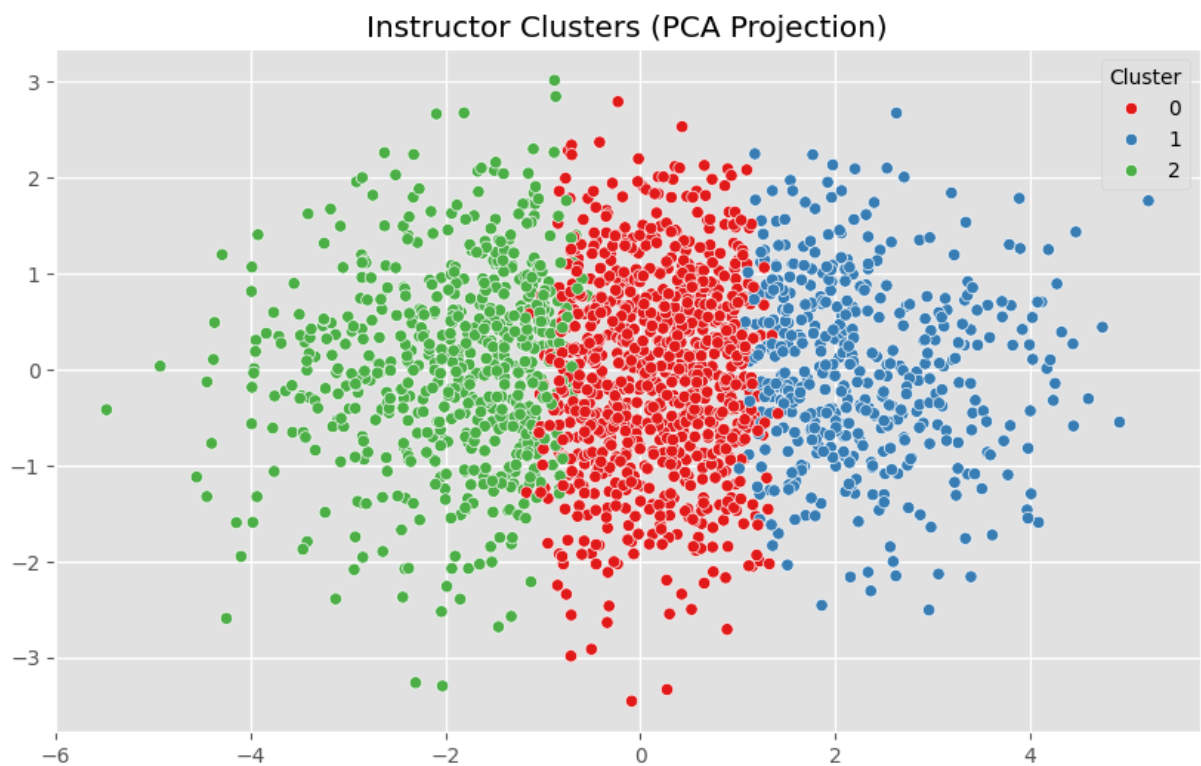
Silhouette Score : 0.108

```
In [125... plt.figure(figsize=(10,6))

sns.scatterplot(
    x=X_pca[:,0],
    y=X_pca[:,1],
    hue=df["Cluster"],
    palette="Set1"
)

plt.title("Instructor Clusters (PCA Projection)")

plt.show()
```



```
In [126... cluster_summary=df.groupby("Cluster")[num_cols].mean()

cluster_summary
```

	completion_rate	avg_score_improvement	avg_quiz_score	dropout_rate	avg_w
Cluster					
0	0.618009	27.329860	78.485429	0.379549	
1	0.793725	31.451231	84.743877	0.202414	
2	0.437760	23.303533	72.104550	0.561287	

```
In [133... import sys
!{sys.executable} -m pip install plotly
```

```
Collecting plotly
  Using cached plotly-6.8.0-py3-none-any.whl.metadata (9.0 kB)
Collecting narwhals>=1.15.1 (from plotly)
  Using cached narwhals-2.23.0-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: packaging in ./my-ml-project/venv/lib/python
3.12/site-packages (from plotly) (26.0)
Using cached plotly-6.8.0-py3-none-any.whl (9.9 MB)
Using cached narwhals-2.23.0-py3-none-any.whl (458 kB)
Installing collected packages: narwhals, plotly
  _____ 2/2 [plotly]—— 1/2 [plotly]
Successfully installed narwhals-2.23.0 plotly-6.8.0

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: /home/rajnish-kumar-sadhan/my-ml-project/venv/bin/p
ython -m pip install --upgrade pip
```

```
In [134... import plotly.express as px
import plotly.graph_objects as go
```

```
In [135... fig = px.scatter(
    df,
    x="completion_rate",
    y="avg_quiz_score",
    color="Cluster",
    hover_data=[
        "instructor_id",
        "course_id",
        "avg_feedback_score"
    ],
    title="Completion Rate vs Quiz Score"
)

fig.show()
```

```
In [136... fig = px.histogram(
    df,
    x="avg_feedback_score",
    nbins=25,
    color="Cluster",
    title="Distribution of Feedback Score"
)
```

```
fig.show()
```

```
In [137... fig = px.imshow(  
    df[num_cols].corr(),  
    text_auto=True,  
    color_continuous_scale="RdBu_r",  
    title="Correlation Heatmap"  
)  
  
fig.show()
```

```
In [138... fig = px.box(  
    df,  
    y="avg_feedback_score",  
    color="Cluster",  
    title="Feedback Score by Cluster"  
)  
  
fig.show()
```

```
In [139... ranking = (  
    df.groupby("instructor_id")  
    .agg({  
        "performance_score": "mean",  
        "avg_feedback_score": "mean",  
        "completion_rate": "mean"  
    })  
    .sort_values("performance_score", ascending=False)  
)  
  
ranking.head(20)
```

Out[139...

	performance_score	avg_feedback_score	completion_rate
instructor_id			
I_010	0.922606	4.552640	0.940783
I_037	0.902874	4.532852	0.923555
I_018	0.884033	4.635637	0.867026
I_091	0.880616	4.490270	0.897119
I_105	0.870072	4.507488	0.863880
I_005	0.854363	4.202516	0.859747
I_118	0.852714	4.641986	0.820753
I_025	0.839192	4.374507	0.822837
I_013	0.835538	4.450021	0.805156
I_051	0.827475	4.479606	0.793270
I_102	0.826099	4.469001	0.743264
I_103	0.818371	4.439677	0.773417
I_087	0.817141	4.375858	0.776333
I_058	0.815136	4.398554	0.774506
I_034	0.811549	4.296384	0.763323
I_015	0.806809	4.521249	0.780741
I_003	0.806342	4.450034	0.768447
I_030	0.794389	4.398047	0.738472
I_066	0.792258	4.286561	0.749361
I_002	0.788755	4.343002	0.730874

In [140...

```
course_rank = (
    df.groupby("course_id")
    .agg({
        "completion_rate": "mean",
        "avg_feedback_score": "mean",
        "avg_quiz_score": "mean"
    })
    .sort_values("completion_rate", ascending=False)
)

course_rank
```

Out[140]...

	completion_rate	avg_feedback_score	avg_quiz_score
course_id			
C_20	0.639422	4.286632	78.828736
C_14	0.639152	4.193211	78.071778
C_05	0.630338	4.237671	76.924427
C_21	0.625749	4.192291	78.740188
C_06	0.622184	4.315455	79.016035
C_24	0.617382	4.274605	79.136514
C_25	0.616483	4.252313	80.003882
C_01	0.611400	4.218196	77.458578
C_11	0.607435	4.106599	80.611406
C_17	0.607274	4.167867	76.594139
C_12	0.606735	4.235956	77.824460
C_02	0.601831	4.191351	78.197494
C_18	0.599867	4.157282	77.770557
C_07	0.598774	4.209100	76.570166
C_04	0.597575	4.225682	78.883837
C_23	0.593806	4.205706	77.109156
C_10	0.592842	4.220344	78.212515
C_13	0.592635	4.120885	78.169013
C_03	0.592443	4.188071	77.091312
C_15	0.590982	4.104935	78.112734
C_19	0.589855	4.161175	76.714415
C_16	0.580683	4.161754	76.146757
C_22	0.574229	4.187142	78.160854
C_08	0.571638	4.300149	76.880615
C_09	0.561902	4.240767	77.106312

In [141]...

```
ranking.to_csv("Instructor_Ranking.csv")  
course_rank.to_csv("Course_Ranking.csv")  
cluster_summary.to_csv("Cluster_Summary.csv")
```

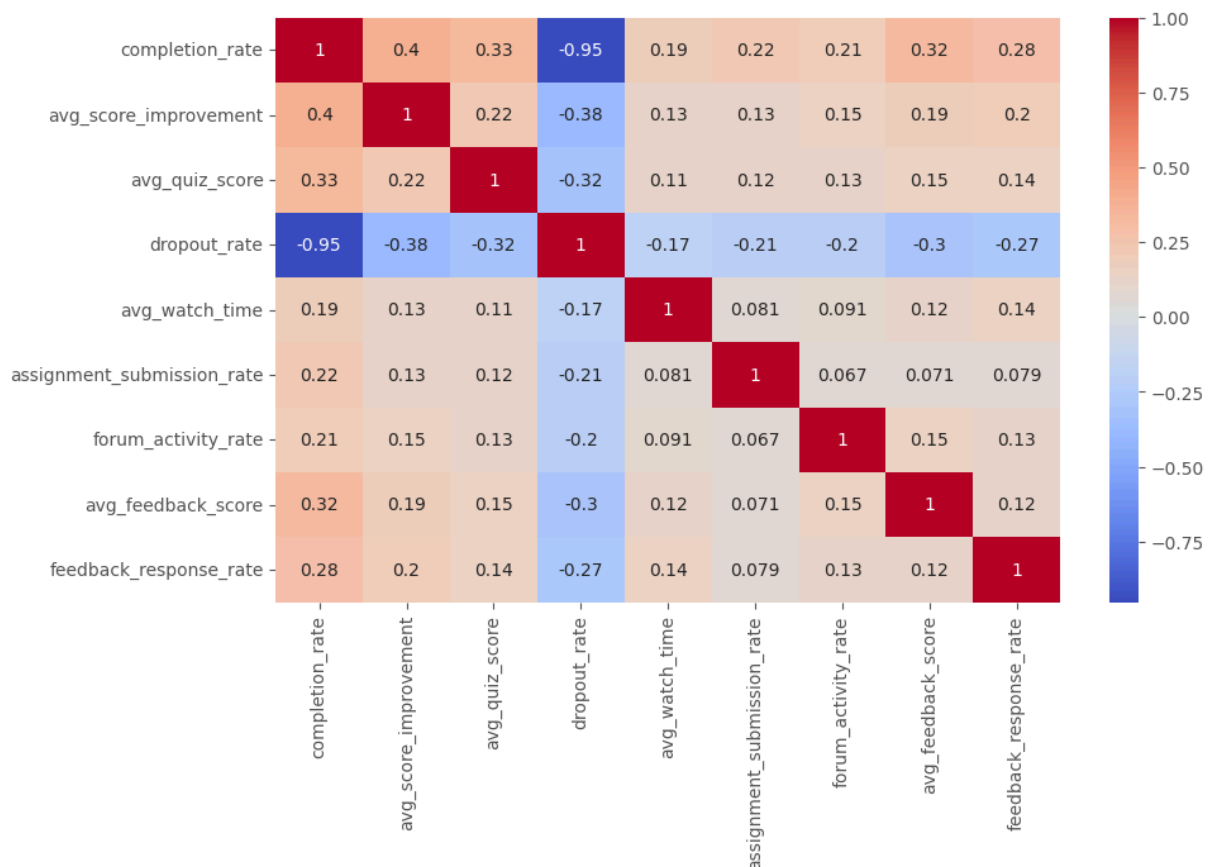
```
df.to_csv("Final_Dataset_With_Clusters.csv",index=False)
```

```
In [142]: plt.figure(figsize=(10,6))

sns.heatmap(
    df[num_cols].corr(),
    annot=True,
    cmap="coolwarm"
)

plt.savefig("Correlation_Heatmap.png")

plt.show()
```



In []: Executive Summary

This project analyzed instructor effectiveness using student engagement and The dataset consisted of 2,000 instructor-course observations across multiple A comprehensive exploratory data analysis was performed to understand patterns Principal Component Analysis (PCA) was used for dimensionality reduction, which The findings provide actionable recommendations for improving teaching effectiveness

In []: Future Scope

- Build a predictive model when labeled data becomes available.
- Develop a recommendation system for instructors.
- Deploy the analysis as a Streamlit dashboard.

- Integrate Power BI/Tableau **for** executive reporting.
- Perform time-series analysis **if** historical data **is** available.

In []: Conclusion

This project successfully analyzed instructor effectiveness using educational data. The dataset was clean, **with** no missing **or** duplicate records. Exploratory Data Analysis revealed meaningful relationships between completion rates and instructor ratings. PCA reduced dimensionality **for** visualization, **and** K-Means clustering identified distinct instructor groups. These insights can help educational institutions improve teaching quality, reduce costs, and enhance student outcomes. Overall, the analysis demonstrates how data-driven approaches can enhance educational performance.

In []: Thank you **for** reviewing this assignment.